DEEP RESEARCH REPORT II · TECHNICAL SURVEY

The Path to Magnetar-Class Proving Efficiency

What the 2024–2026 proof-system literature contributes to the magnetar reference implementation: forty-plus verified papers across zkVM systems, sum-check acceleration, commitments and accumulation, hash arithmetization, and GPU proving — mapped item by item onto the measured anatomy of the stack, with ranked implementation items, quantified projections, and a negative-results register.

The magnetar reference implementation project

github.com/idrees2516/magnetar — extends the October 2026 deep-research report (v1.0); cites IACR ePrint 2026/2423 and the 2026 frontier systems.

Table of Contents

1. Executive Summary 3

2. Baseline and Gap Analysis 6

2.1 Decomposing the 98x . . . 7

2.2 The proof-size anatomy . . . 7

2.3 The 2024-2026 throughput landscape . . . 8

3. The zkVM Systems Landscape 9

3.1 Jolt and the lattice turn . . . 10

3.2 Airbender: the whole-pipeline GPU template . . . 11

3.3 SP1 Hypercube, Flock, and the Jagged idea . . . 11

3.4 ZisK: segmentation as a first-class design . . . 11

3.5 Ziren: the arithmetization census that explains the gap . . . 12

3.6 OpenVM 2.0 and Valida: execution as the co-bottleneck . . . 12

3.7 The seven portable patterns . . . 13

4. Sum-Check Acceleration and Binary-Field PIOPs 13

4.1 The 2025/1117 + 2026/587 lineage: small-value, eq-tensor, streaming . . . 14

4.2 Constraint packing and the univariate skip . . . 14

4.3 The AG-sumcheck: code-theoretic objects in the dominant round . . . 15

4.4 Flock's machinery, beyond the benchmarks . . . 15

4.5 The 2026 refinements: monomial basis, trace packing, tower arithmetic . . . 16

4.6 Round-count and the two structural items . . . 16

5. Commitments, Accumulation and Proof Size 17

5.1 Fast encoders: Bolt, SPIN, and the code families behind them . . . 18

5.2 The Reed-Solomon line and the FRI-Binius wrap alternative . . . 18

5.3 The accumulation problem is solved in the literature . . . 19

5.4 Opening compression: Canopy and the batch-query kit . . . 20

5.5 Soundness repricing and the digest question . . . 21

5.6 The lattice fallback shelf . . . 21

6. The Verifier-Guest Wrap: Hash Arithmetization and Witness Compression 23

6.1 Why Keccak is cheap in characteristic two . . . 24

6.2 The GKR stretch: Keccacheck . . . 24

6.3 The BLAKE3 Merkle-path sub-circuit . . . 25

6.4 The minimal wrap circuit . . . 25

6.5 Keep the standard hashes . . . 26

7. Memory Checking and Lookup Arguments 26

7.1 The lookup landscape . . . 27
8. Folding, IVC, PCD and the Stateless Layer 28
8.1 PCD plumbing and distributed Fiat-Shamir . . . 28
8.2 The stateless layer and the 2^22-chunk validator . . . 29
9. GPU and Hardware Acceleration 29
9.1 The pipeline pattern, made concrete . . . 30
9.2 The tensor-core spike . . . 31
9.3 Toolchain reality . . . 31
9.4 The bandwidth roofline and the governor . . . 31
10. The Integrated Implementation Roadmap 32
10.1 The arithmetic, stated . . . 33
10.2 What changes in the repository's documents . . . 34
11. Risk Register and Negative Results 34
11.1 Soundness constraints that bind . . . 35
11.2 Deliberate rejections . . . 35
12. Consolidated Bibliography 36

1. Executive Summary

This report is the second deep-research pass on the magnetar reference implementation (the Rust binary-field zkVM modeled on the Magnetar paper, IACR ePrint 2026/2423). The first pass (October 2026, v1.0) established the competitive position against Jolt-Akita, the structural cost model of our 598 KiB proofs, and the recursion and folding landscape. This pass answers a narrower and harder question: **which techniques from the 2024-2026 proof-system literature, exactly, must be implemented for this stack to operate at the Magnetar paper's own efficiency level** - 39 ms and 1.7M proven cycles per second at a 2^{16}-cycle ERC-20 mixed block, rising to 612 ms and 6.9M cycles per second at 2^{22}, with 377-596 KiB single proofs and 8-11 ms verification - and then beyond it, toward the repository's stated 150M cycles-per-second GPU milestone. It is a research deliverable: nothing here has been implemented, and every recommendation is gated on explicit approval before implementation begins.

98x
wall-clock gap at 2^16: our 3.8-4.3 s vs the paper's 39 ms (erc20-mixed-block)

~15x
of that gap is hardware: 12 M4 performance cores with ~5x faster CLMUL vs 4 cloud vCPU

~6.5x
the algorithmic residual this report attacks: per-core, per-round, per-row constants

The survey found that the ground moved under the entire field between 2024 and 2026, and it moved in directions that favor this stack's architecture. Five developments dominate. First, every credible frontier prover moved to GPUs end-to-end (Airbender at 21.8M cycles/s per H100; Ziren at 12M per RTX 5090 scaling to 75.8M on eight; OpenVM 2.0 at 11.4M per 5090 scaling to 139M on sixteen), which both raises the bar and hands us a proven engineering template. Second, sum-check prover algebra advanced sharply - the small-value and eq-tensor optimizations of ePrint 2025/1117 and 2026/587 deliver measured 2-3x on exactly the Spartan-class sum-checks that consume 63% of our prover time, and the algebraic-geometry sum-check of ePrint 2026/2494 attacks the most expensive round with code-theoretic objects. Third, the open problem blocking unbounded accumulation for our commitment code was solved in the literature: WARP (2025/753) gives linear-time, unbounded-depth, hash-based accumulation for **any linear code over a sufficiently large field**, and Khatam (2024/1843) supplies the proximity gap for multilinear folding over all linear codes - GF(2^{128}) and our affine-substitution codewords qualify. Fourth, Flock (2026/1329) and Keccacheck (2025/1764) published concrete arithmetizations that bring standard hashes down to 41.6K and effectively 3.2K committed witness bits per Keccak permutation - which is precisely the missing piece of our verifier-guest wrap circuit, whose naive cost estimate of ~132K bits per permutation breaks the 2^{28}-2^{29} witness budget. Fifth, opening compression (VCR/Canopy, 2026/2790) measured -50% proof size on Bolt-code systems, which converts our 128 KiB composed-projection into a credible 90-105 KiB at 120-bit provable soundness.

The resulting implementation portfolio is ranked in Table 1. The top three items are deliberately cheap: they are engine-level algebraic changes (the eq-tensor optimization, binary-tower constraint packing, and the window pass extended to the Shout high-degree sum-check) with published measured anchors, no protocol changes, and byte-identical round messages - the same discipline every 0.9.x kernel release of this repository already follows. The middle of the table restructures the arithmetization (per-opcode chips with a published

cost census, jagged mixed-height commitments, per-chip trace splitting), and the bottom builds the two structural additions the paper's scaling path requires: the verifier-guest wrap circuit and the unbounded accumulation layer. Everything is mapped to a specific crate, a specific prover phase from our measured anatomy, and a specific external measured anchor, and every projected gain is labeled as a projection rather than a measurement.

#	Item (source)	Our layer	Measured anchor (theirs)	Projected gain (ours)	Effort	Verdict
1	Eq-tensor optimization + small-value rounds (2025/1117, 2026/587) - never materialize eq(r,x); fold it variable-by-variable	magnetar-sumcheck engine + both Spartan outers	2-3x on Spartan-in-Jolt; >10x in the memory-bound regime; 25%/10% e2e on prime fields	~1.3-1.6x wall-clock (Spartan 29% + discharge 34% are sum-check bound)	S-M	Must - first
2	Binary-tower constraint packing (2024/1038) - pin challenge coordinates in GF(2^8), cut eq pre-compute ~128x	Same sites as #1	Cuts eq precompute by log|F| in the zerocheck shape; char-2 specific	Bundled with #1	S	Must - bundle
3	Window pass extended to the Shout high-degree sum-check (2026/587 technique 2)	window.rs shape generalization + magnetar-memory::shout	1.7-2.2x on exactly this sub-protocol in Jolt	~5% e2e (projection: Shout-class rounds ~1/3 of the 32% memory block)	M	Must
4	Twist-and-Shout 2025 revision audit - small-characteristic one-hot corrections	magnetar-memory::twist	Twist: 3.7x fewer nonzeros, 1.7x fewer mults vs Spice; 3x prover speedup inside Jolt	1.1-1.5x on our 32% Twist phase	S-M	Must
5	Per-opcode chips + instruction frames + a published cost census (Ziren 2026/2330)	magnetar-cpu circuit layer	33-37 committed cells per common op vs our padded 512 rows/cycle; one 163-cell DivRem row vs 8-step sequences	Up to 2.5-3x on all arithmetization phases (medium confidence - normalize units first)	L	Must (census first)
6	Jagged mixed-height commitment + LSB-paired folding (SP1 Jagged PCS; Ziren 10.3)	MosaicPCS layout + magnetar-router join	-28% Ziren measured from pairing alone; 2x shard capacity	Bigger segments amortize per-segment roots; less padding bytes	M	Must

#	Item (source)	Our layer	Measured anchor (theirs)	Projected gain (ours)	Effort	Verdict
7	The wrap circuit: Keccak at 22.4-41.6K bits/perm + BLAKE3 paths (Flock 2026/1329)	new wrap-guest circuit in magnetar-zkvm	Flock: 78.4K all-wires / 41.6K only-ANDs committed bits per perm, measured	Unlocks the 100-160 KiB composed proofs; wrap witness 2^27.7-2^28.6 vs naive 2^30.2	M	Must (H2)
8	Keccacheck GKR batch sub-argument (2025/1764)	char-2 sum-check sub-protocol inside the wrap	<4,000 constraints per permutation at batch >= 512; >10x cost reduction	Committed hash witness down to ~2^25 bits total	M-L	Must (stretch)
9	Canopy/VCR opening compression (2026/2790)	magnetar-pcs::eval + wrap wiring	-50% proof size, 3x fewer verifier hashes on Bolt codes; -20% on RS	Composed proofs 128 -> ~90-105 KiB	M	Must
10	WARP-shaped unbounded accumulation over our code (2025/753 + 2024/1843)	magnetar-zkvm::wrap accumulation layer	Linear prover, log verifier, unbounded depth, any linear code over a large field	Removes the BMNW24 epsilon-loss and depth ceiling at any tree depth	M-L	Must (research import)

Table 1: The top-ten implementation items, ranked by expected gain x confidence / effort.

Composed, the portfolio projects as follows (all projections, arithmetic shown in Chapter 10): the Phase-0 engine items multiply to roughly 1.35-1.6x wall-clock on the current 4-vCPU host; the Twist revision and the arithmetization restructure together carry the per-core rate from today's ~4.3K proven cycles per second per vCPU toward 10-13K, which is within 2-3x of the paper's own hardware-normalized per-core operating point (~28K per core on our Xeon class after the 5x CLMUL discount) - with the remaining factor living in per-kernel constants that this repository's release ladder has historically closed at 1.5-2x per pass. The decisive lever, however, is the GPU: 96% of our prover time is in GPU-able phases, and at the 15-30x kernel gains the 2026 systems measured, one datacenter GPU lands at 0.4-0.7M cycles/s on our current arithmetization and 2-6M after the Phase-1 restructure - at which point cluster scaling (3.1-3.3x per four GPUs, Ziren-measured) puts the 150M program target within the envelope the frontier systems already occupy. On proof size, the path is 596 KiB unwrapped, 128 KiB at the dual-rate wrap (both measured on this repository), then 90-105 KiB with Canopy-class opening compression, holding 120-bit **provable** soundness - inside the paper's composed band and within 1.2-1.5x of Jolt-Akita's lattice proofs, with only standard hashes underneath.

The report also carries a negative-results register (Chapter 11), because a survey that cannot say no is useless. We reject 16-byte digests (they cap collision-type security near 64 bits against a 2^{60}-query adversary; the conservative 120-bit bound needs 240-bit digests), we keep SHA3 and BLAKE3 rather than swapping in Poseidon2b or Vision (the Flock and Keccacheck results make standard hashes cheap enough to preserve the

paper's post-quantum stance), we do not fork the stack toward lattice folding (Neo/SuperNeo would require a second proving stack over Z_q), and we correct five errors in our own leads list - including that FRI-Binius is ePrint 2024/504 rather than a 2025 paper, that the AG-sumcheck's 2.36x is the skip-round figure rather than end-to-end, and that Airbender is ZKsync's prover, not a GPU port of Jolt.

2. Baseline and Gap Analysis

Everything in this report is anchored to two measured baselines: the Magnetar paper's Table 6 (eprint 2026/2423, section 6, the optimized system on 12 performance cores of an M4 Max with 128 GiB) and this repository's own benchmark harness (the erc20-mixed-block workload, proved and verified end-to-end, on a 4-vCPU EPYC instance with AVX-512 and VPCLMULQDQ). The paper's abstract additionally reports 2^{22}-cycle executions at 7.2-8.7M cycles/s at 120 bits of provable security; we use the Table-6 figures as the primary reference because they carry the full workload shape. Our numbers are from the fw-bound-se group-fold and split-ring releases at the 2^{13}-cycle segment operating point, with every row verified, and they already exhibit the paper's central structural claims: linear-in-T proving, logarithmic communication, and a structured verifier.

System	Trace T	Prove	Cycles/s	Peak RSS	Proof	Verify	Host	Source
Paper	2^16	39 ms	1.7M	0.2 GiB	377 KiB	8 ms	12 M4 cores	Table 6
Paper	2^18	61 ms	4.3M	0.5 GiB	462 KiB	9 ms	12 M4 cores	Table 6
Paper	2^20	174 ms	6.0M	1.6 GiB	531 KiB	10 ms	12 M4 cores	Table 6
Paper	2^22	612 ms	6.9M	6.4 GiB	596 KiB	11 ms	12 M4 cores	Table 6
This repo	2^16	3,821-4,254 ms	15.4-17.2K	0.84-1.09 GiB	one shared tail	19-43 ms (2^8-2^10 shape)	4-vCPU EPYC	BENCHMARKS.md
This repo	2^19	47.3 s	11.1K	3.63 GiB	one shared tail	-	4-vCPU EPYC	BENCHMARKS.md
This repo	2^20	116.2 s	9.0K	7.06 GiB	one shared tail	-	4-vCPU EPYC	BENCHMARKS.md

Table 2: The paper's Table-6 targets versus this implementation's measured results.

2.1 Decomposing the 98x

At 2^{16} the wall-clock gap is 3,821-4,254 ms against 39 ms, roughly 98-109x. Hardware accounts for most of it. The paper's M4 Max performance cores run CLMUL operations at roughly 5x the rate of this host's measured 11.5 ns/mul Xeon cores, and the paper uses 12 such cores against our 4 vCPUs: a combined hardware factor of approximately 15x. That leaves an algorithmic-and-constants residual of about 6.5x, and that residual is what this report attacks. Per-core, the paper proves ~142K cycles per second per M4 core at 2^{16}; hardware-normalized to our Xeon class that is ~28K per core, while we sustain ~4.3K per vCPU today. The residual decomposes across the same phases the repository's MAGNETAR_TIME=1 instrumentation already measures, and each research lever in this report lands on a specific phase - the mapping is Table 3, and it is the organizing device for Chapters 3 through 9.

Prover phase (share of wall clock)	Share	The levers that attack it (chapter)
Execute + witness generation	4%	AOT tracer + GPU witness expansion, the OpenVM 3.8 GHz pattern (Ch. 3); the GPU tracer governor (Ch. 9)
Commit (encode + Merkle)	5%	Bolt/SPIN-class encoders at (3+eps)N adds + (1+eps)N hashes (Ch. 5); tensor-core encode (Ch. 9)
Virtual Spartan (outers + inner)	29%	Eq-tensor + constraint packing + small-value rounds (Ch. 4); monomial-basis sum-check (Ch. 4)
BinaryTwist memory (registers + RAM)	32%	Twist-and-Shout 2025 small-characteristic revision (Ch. 7); select-folds on bit-valued tables (repo lineage)
BinaryRouter (join provers)	3%	Join extraction pre-pass (repo item); jagged layout (Ch. 3)
Bus Shout + selectors + cross-step	1%	Window extension to the Shout high-degree sum-check (Ch. 4)
Mosaic discharge (ring switch + eval tail)	34%	Canopy/VCR opening compression; WARP accumulation; FRI-Binius-style wrap variant (Ch. 5)

Table 3: The measured prover phase anatomy (T = 2^8) and where each research lever lands.

2.2 The proof-size anatomy

Proof bytes live in a different place than prover time, and the distinction matters for prioritization. At 2^{22} the paper's 596 KiB proof is dominated by the evaluation argument - approximately 84% (~500 KiB) is the correlated Merkle-path serialization of roughly 40 queries times three authentication paths times (n+k) 32-byte digests, plus tails and level roots - while the sum-check machinery accounts for the remaining ~16% (~95 KiB). This repository's dual-rate profiles already measure the size dial on the same statement: the evaluation tail runs 62.8 KiB at FastChunk (rate 1/2, 40 queries) down to 27.3 KiB at DeepWrap (rate 1/16, 14 queries), and the wrapped configuration lands at 36.5 KiB, a 44% tail reduction. The composed projection at 2^{22} is 128 KiB (216 KiB at FastChunk down-ported to DeepWrap), already inside the paper's 100-160 KiB composed band; Chapter 5 adds the two published levers - opening compression and unbounded accumulation - that hold that size at any tree depth and push it toward ~90-105 KiB.

2.3 The 2024-2026 throughput landscape

Figure 1 places both baselines in the 2026 field. Three facts stand out. First, the paper's 6.9M cycles/s CPU operating point - our target - sits mid-pack among the current GPU systems, so reaching it is a threshold, not a summit. Second, every system above the paper's line runs on GPUs, and the two cluster points (Ziren at 75.8M on 8 GPUs, OpenVM 2.0 at 139M on 16) show that the 150M program milestone is a cluster-scale statement in the class the frontier already occupies. Third, the vertical distance between our CPU bar and the paper's bar is the 98x this report decomposes; the horizontal structure of the chart - systems clustering at 11-22M per single GPU - is the evidence base for the GPU phases of Chapter 9.

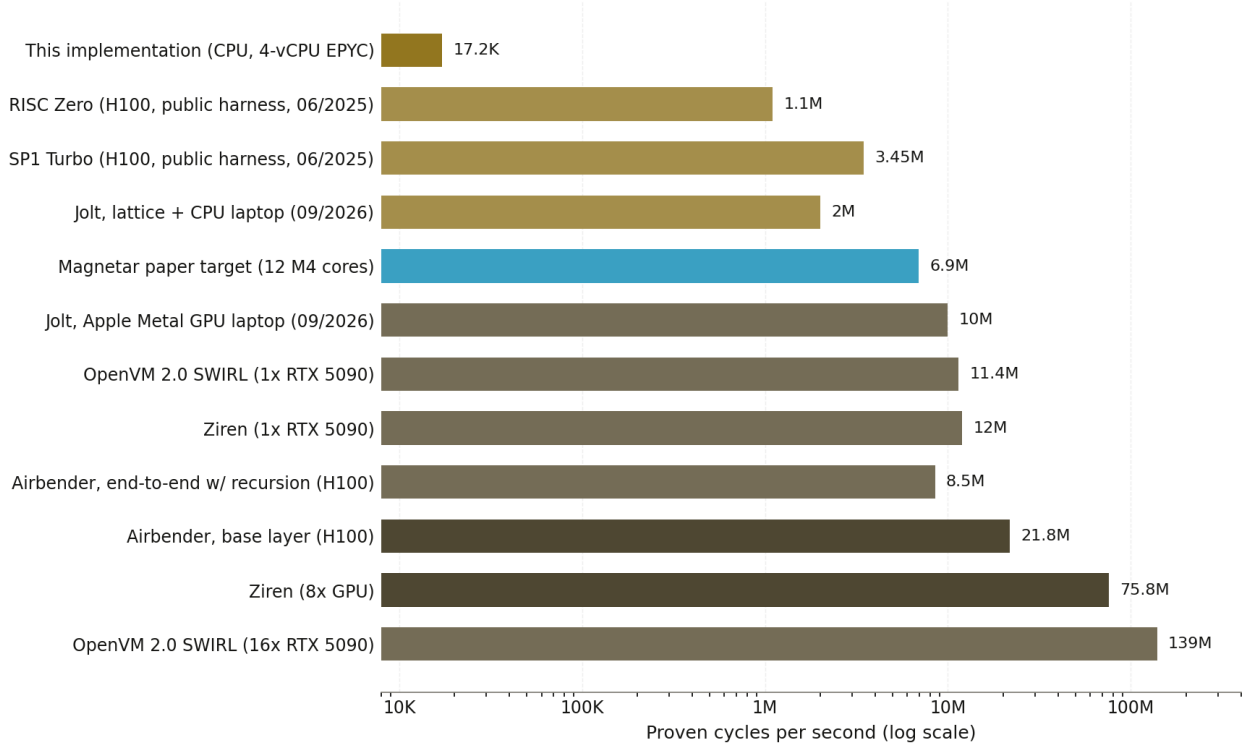


Figure 1: The 2024-2026 proving-throughput landscape (proven cycles per second, log scale). Gold = this implementation; blue = the Magnetar paper target; olive = published system measurements (sources: Chapters 3 and 9). The cluster points are Ziren on 8 GPUs and OpenVM 2.0 on 16 GPUs.

One definitional caution before the survey proper. Cross-system cycles-per-second comparisons carry heavy caveats: trace density (cycles per transaction) differs by workload and ISA, security parameters differ (100 vs 120 bits, provable vs conjectured), and base-layer numbers exclude recursion while end-to-end numbers include it. We therefore use each system's own reported numbers only to extract **techniques and their measured effects**, never as a leaderboard; where units are not comparable - our 512 rows per cycle versus Ziren's 58.9 committed cells per instruction - the report says so explicitly and orders the normalization work before any conclusion is drawn from the ratio.

3. The zkVM Systems Landscape

Four research agents opened roughly forty primary sources for this cluster: the Jolt lineage and its 2026 lattice turn, ZKsync's Airbender and its October 2026 Prime release, Succinct's SP1 Hypercube and Flock, Polygon's ZisK, ZKM's Ziren, Axiom's OpenVM 2.0, Lita's Valida with the vApps results, and RISC Zero. Table 4 summarizes the field; the subsections then extract, for each system, the techniques that transfer to a GF(2^{128}) hash-based sum-check stack, with their measured numbers. Two of our going-in assumptions were corrected by the sources and are flagged here so the roadmap inherits the correction: Airbender is ZKsync/Matter Labs' own STARK-lineage prover, not a GPU port of Jolt, and the widely-repeated 'ZisK 1.5 GHz tracer' claim could not be verified in any ZisK whitepaper, documentation, or release note we could open.

System	Architecture	Field / commitments	Reported throughput	Proofs	Notable
Jolt (a16z)	Lookup-singularity RV64IMAC; sum-check + Twist/Shout; Spartan inner; RV64 rewrite 2025	EC (Zeromorph/HyperKZG); 2026 lattice (Akita)	>1M c/s 32-core CPU (07/2025); >1.5M RV64 (10/2025); 09/2026 lattice: >2M CPU, >10M MacBook Metal GPU	~50 KB; lattice <100 KB	The 2025/1117 sum-check work was 'essential for Spartan within Jolt'
Akita	Module-SIS lattice PCS; commitment time proportional to nonzero entries	Lattice (Module-SIS)	(inside Lattice-Jolt)	58-68 KB	ePrint 2026/1983; built for Twist/Shout sparsity
Airbender (ZKsync)	RV32I+M AIR over Mersenne31; DEEP-ALI + FRI; delegation; RAM-backed registers; CPU traces, GPU proves	Hash-based (FRI)	21.8M c/s base / 8.5M e2e per H100 (06/2025); Prime (10/2026): 99% of 7,200 mainnet blocks <10 s on one RTX PRO 6000 Blackwell	not published per-GPU	Five-stage pipeline; >=22 GB VRAM; MIT-licensed GPU prover
SP1 Hypercube	First all-multilinear zkVM; Jagged PCS (pay-per-use heights); chip + cross-table-lookup design	Hash-based	99.7% of mainnet blocks <12 s on 16x RTX 5090 (11/2025)	cluster-sized	62 RISC-V opcodes formally verified vs the Sail spec; mainnet Feb 2026
Flock (Succinct)	Batch-R1CS over F2: zerocheck + lincheck; ring-switching to F(2^128); Ligerito MLPCS	Hash-based (Ligerito)	82.1K BLAKE3/s per M4 core; 660K/s on 10 cores; 30.7K Keccak-f/s	200-558 KiB at ~1.5x2^14 perms	ePrint 2026/1329; the batch-hash SNARK of record
ZisK (Polygon)	STARK over Goldilocks; multi-chip AIR + 5 LogUp buses; per-chip trace splitting; LtHash order-free aggregation	Hash-based	no public c/s figure (spec whitepaper); v1.4.0-alpha 10/2026	(PLONK wrap optional)	OP-Mainnet census: MAIN 59.5%, OPCODES 11.3%, PRECOMPILES 16.1%, MEMORY 11.6%

System	Architecture	Field / commitments	Reported throughput	Proofs	Notable
Ziren (ZKM)	MIPS32 CPU-less chips; per-shard one lookup + one zerocheck + jagged sum-check; one batched WHIR proof	Hash-based (WHIR)	12.0M c/s per RTX 5090; 75.8M on 8 GPUs (ePrint 2026/2330)	93.5-bit composite over a 149-proof tree (honest accounting)	104 determinism theorems in Lean 4; 58.9 committed cells + 25.8 bus pairs per instruction
OpenVM 2.0	SWIRL: WHIR-backed multilinear; Zerocheck + LogUp-GKR + Stacked Reduction; Univariate Skip; write-once-memory recursion	Hash-based (WHIR)	11.4M c/s per RTX 5090; 139M on 16; p99 11.8 s over 1,000 mainnet blocks	<300 KB at 100 bits	AOT execution at 3.8 GHz (25x over interpreter); Lean-verified RV32IM
Valida / vApps	STARK over 31-bit field; no registers or stack - memory-direct; LLVM backend	Hash-based	vApps: native Rust guests up to 197x fewer cycles; GPU up to 30x	recursion compresses 230-300x	ePrint 2025/1110 (DSL); arXiv 2504.14809 (vApps)
RISC Zero	STARK RV32IM	Hash-based	1.1M c/s per H100 (ZKsync public harness 06/2025)	-	no newer self-reported throughput found this survey

Table 4: The 2024-2026 zkVM proving-systems field (their own reported numbers).

3.1 Jolt and the lattice turn

Jolt matters to us for two measured facts rather than its architecture. The first is that the 2025/1117 sum-check optimizations - small-value rounds and the eq(r,x) dot p(x) structure - were developed and measured inside Jolt's Spartan sub-protocol, where they deliver 2-3x and are described by the authors as essential; our Virtual Spartan phase is the same protocol shape in characteristic two, which is why Chapter 4 ranks the eq-tensor port as the single highest-leverage cheap item we can buy. The second is the September 2026 lattice release (Jolt + Akita, ePrint 2026/1983): by replacing the elliptic commitment with a Module-SIS PCS whose commitment time is proportional to nonzero entries - explicitly designed for Twist-and-Shout's near-zero-sparse polynomials - a laptop CPU exceeds 2M cycles/s and an Apple Metal GPU prover exceeds 10M, with proofs under 100 KB and prover memory cut from 300 to 200 bytes per cycle. That is the strongest published evidence that commitment-layer sparsity, not PIOP structure, is the next battleground - and it is the competitive frame our one-hot components and systematic-code sparsity exploitation (already on the Horizon-3 list) should be judged against. We cannot copy Akita without abandoning the hash-only posture, but its pay-per-nonzero discipline is the right target profile for our sparse one-hot commitments.

3.2 Airbender: the whole-pipeline GPU template

Airbender is the single most relevant systems template for our GPU milestone because it publishes both a reproducible baseline and its pipeline shape. The June 2025 numbers - 21.8M cycles/s base layer and 8.5M end-to-end with recursion on one H100, against SP1 Turbo at 3.45M and RISC Zero at 1.1M in the same public harness - were measured on Fibonacci, and the Ethereum-block figures (~17 s pre-recursion, ~35 s end-to-end) came with the pool economics ($0.0001 per transfer). The architecture is the lesson: the CPU only runs RISC-V simulation and tracing; the GPU does circuit-specific witness generation (a dedicated gpu_witness_eval_generator mirrors the CPU implementation) and all five proving stages, with an asynchronous allocator and pipelining that overlaps host-device transfers with compute, resident data across stages, and CUDA graphs. The October 2026 Prime release swapped the stack to GKR sum-check + WHIR commitments over BLAKE2s and proved 99% of 7,200 consecutive Ethereum mainnet blocks in under 10 seconds each on one RTX PRO 6000 Blackwell - leading the ETHproofs leaderboard. For magnetar, the import is the pattern, not the code: our phase anatomy shows 96% of prover time in GPU-able phases, and Chapter 9 translates the Airbender pipeline into our phases with the Chimera residency discipline and the cuPQC kernel-fusion toolchain.

3.3 SP1 Hypercube, Flock, and the Jagged idea

SP1 Hypercube (mainnet February 2026) contributes two transferable ideas. The Jagged PCS makes commitment cost proportional to what each chip actually uses - mixed-height columns, each folding out after log2 of its own height - instead of padding everything to a common power of two; this is the production ancestor of the jagged sum-check Ziren measures at -28%. And the 2026 release completed formal verification of all 62 RISC-V opcodes against the Sail specification (with Nethermind and the Ethereum Foundation), which - with OpenVM's Lean-verified RV32IM and Ziren's 104 Lean determinism theorems - makes ISA formal verification table stakes for production zkVMs; our cross-step chain and its twelve tamper classes are the natural target for the same treatment. Flock, covered in depth in Chapters 4 and 6, is Succinct's June 2026 batch-hash SNARK (with Bunz, Rothblum, and Wang's ePrint 2026/1329) - the closest published relative of our Mikros layer, and the source of the wrap circuit's witness-compression arithmetic.

3.4 ZisK: segmentation as a first-class design

ZisK's whitepaper and documentation describe the segmentation discipline we should adopt for the tree-IVC layer. Instead of checkpointing the machine every fixed interval - our current 2^{13}-cycle segments, where every chip pads every segment - ZisK runs the program once to completion, then splits **each chip's trace at its own height**, padding only in each chip's tail; continuation buses carry per-chip state across segments, and an opportunistic recursive aggregation tree uses an order-independent LtHash shared challenge so that workers can prove in any order. Their published cost census of an OP-Mainnet block (197M steps: MAIN 59.5%, OPCODES 11.3%, PRECOMPILES 16.1%, MEMORY 11.6%, including 31,068 Keccaks per precompile instance) is the methodology template for our own census. The claim we could not verify - the 1.5 GHz-class tracer sometimes attributed to ZisK - is flagged in Chapter 11; the verified fast-tracer story in the industry is OpenVM's AOT compiler (Section 3.6).

3.5 Ziren: the arithmetization census that explains the gap

Ziren (ePrint 2026/2330, October 2026) is the richest single source in this cluster because it publishes the internals a comparison needs. Its CPU-less chip architecture gives each executed instruction one row in its opcode's chip, with a PC-pair instruction frame and no central CPU table; common operations cost 33-37 committed cells per row against 400+ for a unified table, and DivRem is a single 163-cell row against Jolt's 8-step sequence. Per shard it runs one lookup argument over all buses, one zerocheck over all constraints, and a jagged sum-check over tens of thousands of mixed-height columns, discharged by one batched WHIR proof; LSB-paired GKR folding lets each short chip fold out after log2 of its height, which measured -28% and doubled shard capacity. The GPU kernel anatomy is published too: per shard, LogUp-GKR 42%, WHIR commit+open 19%, zerocheck 16.5% compiled plus 7.1% interpreted, jagged reduction 8% - which tells us exactly where GPU effort pays first (bus arguments and Merkle/encode, then zerocheck). The honest security accounting (93.5-bit composite over a 149-proof tree) is the model for our tree-IVC union bound. The comparison caveat is explicit: Ziren's 58.9 committed cells per instruction are 31-bit-field cells, our 512 rows per cycle are binary relation rows - the units are not directly comparable, which is why recommendation 5 orders a committed-bits-per-cycle census of our own before any ratio is trusted.

33-37
committed cells per common op in Ziren's per-opcode chips, vs 400+ in a unified table

-28%
Ziren's measured shard cost reduction from LSB-paired jagged folding alone

3.8 GHz
OpenVM's AOT-compiled guest execution rate - the verified answer to the tracer bottleneck

3.6 OpenVM 2.0 and Valida: execution as the co-bottleneck

OpenVM 2.0 'Powered by SWIRL' (January 2026, production July 2026) is the systems confirmation of two things this report recommends. Its proof system is the 2026 convergence pattern - WHIR-backed multilinear commitments, Zerocheck + LogUp-GKR + a Stacked Reduction, small-field Univariate Skip, and write-once-memory recursion that aggregates a dynamic number of proofs - and its numbers (11.4M cycles/s per RTX 5090, 139M on sixteen, p99 11.8 s across 1,000 mainnet blocks, proofs under 300 KB at 100 bits) define the frontier our GPU phases are calibrated against. Its execution layer is the second lesson: after the proving pipeline moved to GPU, trace generation became the bottleneck, and the answer was an AOT compiler that single-pass-compiled RISC-V to x86 and executes at 3.8 GHz - 25x over the 150 MHz interpreter, cutting block execution from 1.8 s to 0.5 s. Our tracer (roughly 450K cycles/s inside the 4% execute phase today) is fine until the GPU milestone, at which point it becomes the governor; Chapter 9 carries the AOT-plus-GPU-witness-expansion plan. Valida's vApps result - native Rust guests removing up to 197x cycles versus EVM interpretation, with precompiles accelerating proving over 95% - is the same argument at the workload layer, and it validates our erc20 core-circuit approach of proving the state-transition semantics directly rather than an interpreter loop, wherever the workload allows.

3.7 The seven portable patterns

Distilled from the systems above, seven patterns transfer to this stack, and they recur through the rest of the report: (1) the whole-pipeline GPU port with CPU-trace/GPU-witness split and transfer-compute overlap (Airbender); (2) jagged, pay-per-use mixed-height commitments (SP1, Ziren); (3) per-opcode chips with instruction frames and a published cost census (Ziren); (4) per-chip trace splitting with order-free LtHash aggregation (ZisK); (5) AOT trace generation (OpenVM); (6) formal ISA verification (SP1, OpenVM, Ziren); and (7) small-field discipline end-to-end - every system at the frontier now keeps its arithmetic in the smallest field soundness permits, which for us means the tower below GF(2^{128}) wherever the round message allows it (Chapters 4 and 6). None of these requires abandoning a single design decision of the magnetar stack; all seven are additive.

4. Sum-Check Acceleration and Binary-Field PIOPs

This chapter covers the papers that attack the protocol layer our prover actually spends its time in. Two clarifications of our own leads list came out of the verification pass and are adopted throughout: the round-window lineage is ePrint 2025/1117 ('Speeding Up Sum-Check Proving', Bagad, Dao, Domb, Thaler - the merger of 2024/1046 and 2024/1210) with the extended version at ePrint 2026/587 (Dao, DeStefano, Bagad, Domb, Thaler), and the AG-sumcheck is ePrint 2026/2494 ('Speeding up Sumcheck using Algebraic Geometry', Dalal, Bunz, Hemo, Rothblum, received October 2026). The 2.36x figure in our roadmap is the skip-round figure inside Flock at 2^{32} constraints; the whole-zerocheck figure is 1.65x. The roadmap's 'FRI-Binius (2025/...)' is in fact ePrint 2024/504, and the rumored 'Deep, ePrint 2025/2691' does not resolve - it is dropped from the leads list.

4.1 The 2025/1117 + 2026/587 lineage: small-value, eq-tensor, streaming

The combined paper and its extended version contain the three techniques with the best measured fit to our anatomy. Small-value sum-check observes that when round evaluations live in a small domain - u32-sized integers, or a small subfield inside the extension field, which is exactly our GF(2) inside GF(2^{128}) tower setting - the prover can multiply in the small domain and pay the extension-field cost only at the binding step; our small-value round window (window.rs, the char-2 monomial window with the eq-weighted pack and the select fast-forward, cross-validated byte-identical against the generic engine) already implements the characteristic-two specialization for the Spartan-outer shape. The eq(r,x) dot p(x) optimization exploits the decomposable tensor structure of the equality polynomial so the prover never materializes the 2^n-entry eq table at all - it folds eq variable-by-variable against the other factors - eliminating on the order of 2^{n+1} extension-field multiplications per sum-check; this is the technique our engine still pays for in slab-gather eq passes, and with constraint packing (below) it is the top-ranked item of this entire survey. The authors measured 2-3x on Spartan's first sum-check inside Jolt, 25% and 10% end-to-end on two prime-field systems, more than 10x combined runtime-and-memory reduction on Jolt's Spartan sub-protocol in the extended version, and - the remark that matters most to us - they state the savings are 'potentially even larger when working over binary tower fields.' The streaming prover of the extended version (small-space enumeration, O(kN) time at $O(N^{1/k})$ space per the companion time-space tradeoff paper 2025/1473) is the published primitive our Horizon-2 'streaming within chunks' item was waiting for.

4.2 Constraint packing and the univariate skip

Two earlier techniques ride along in the same code sites. Dao and Thaler's constraint packing (ePrint 2024/1038) pins sum-check challenge coordinates in a small subfield - in their instantiation, coordinates in GF(2^8) inside the GHASH representation of GF(2^{128}), three pinned coordinates per round - which cuts the eq(r,x) pre-computation by roughly log|F| = 128x in the zerocheck/Spartan-outer shape and 'strictly generalizes a folklore technique'; it is characteristic-two-native and cheap to adopt inside the same engine pass as the eq-tensor fold. Gruen's univariate skip (ePrint 2024/108, 'Some Improvements for the PIOP for ZeroCheck') shifts work from the extension field back to the base field by a tunable 2^k factor - prover cost approximately 2^n times (1 + C_G / (2^k C_F)) times (d-1) C_F - and is the ancestor of the skip parameter t_skip that both Flock and Mikros tune over binary towers; our window already embodies its char-2 form for the shared rounds, and the generalization to the remaining skip-eligible rounds is part of recommendation 3.

4.3 The AG-sumcheck: code-theoretic objects in the dominant round

The algebraic-geometry sum-check (ePrint 2026/2494) is the most theoretically ambitious item in this chapter and the one our roadmap's GPU Phase 5 correctly parked. The observation is that all practical sum-check provers work with polynomials, and over binary fields this costs a logarithmic factor in the first, most expensive round - the univariate skip - because the product of two binary messages under Reed-Soloman encoding is no longer binary. Their replacement is a concrete **multiplication code** built from algebraic geometry: a genus-95 curve over F_2 with 64 rational points, constructed as a $(Z/2)^3$ Artin-Schreier cover of a genus-4 curve, carrying 64-bit messages to binary product messages of length 222 with product-code relative distance greater than $1 - 2^{-119}$. The part of the product message the prover sends is more than 3x smaller than with Reed-Solomon. Inside Flock at 2^{32} constraints this 'AG skip' measured 2.36x on the skip round and 1.65x on the whole zerocheck, which was Flock's most expensive component. The applicability caveats are real: the published code is built for 64-bit messages, while our packing width is kappa = 128, so we would either run two 64-bit channels or need the analogous 128-bit-message code (the construction generalizes; the concrete code is not in the abstract); and the engineering is genus-95 curve arithmetic with Artin-Schreier covers - a research-grade dependency. The soundness side is clean: product distance $1 - 2^{-119}$ fits our 120-bit budget. Projection: if skip-eligible rounds are about half of the combined Spartan (29%) plus Twist (32%) block, a 1.65x cut on them is roughly 10-15% end-to-end - worthwhile, but strictly after the eq-tensor and packing items, which cost a fraction of the effort.

4.4 Flock's machinery, beyond the benchmarks

Flock (ePrint 2026/1329, with a public Rust implementation at succinctlabs/flock) is the batch-hash SNARK of record and the closest published relative of our Mikros layer - the same Ligerito MLPCS backend (ePrint 2025/1187), the same ring-switching from GF(2) to $GF(2^{128})$, a Spartan-like zerocheck-plus-lincheck PIOP, and measured rates (82.1K BLAKE3 compressions per second per M4 Max core, 42.1K SHA-256, 30.7K Keccak-f; 8.4x faster than Binius64 on SHA-256; under 250x native cost) that Mikros itself then beat by 1.85-2.55x on batch hashes - a useful calibration of where our implemented stack sits against the published frontier. Four of its internal techniques transfer directly to our engine and wrap circuit: the univariate skip tuned over binary towers; partially deterministic sum-check challenges (the 2024/1038 packing above); deferred modular reduction with evaluation at infinity (the monomial-basis view, Section 4.5); and - most distinctively - the **circuit walker**, which replaces materializing the substituted dense constraint matrix in the lincheck by a single symbolic walk of the hash circuit in reverse topological order, plus an LDE map precomputed as a lookup table over 8-bit chunks of $GF(2^8)$ inside $GF(2^{128})$. The walker is the enabling trick for the wrap's dense-AND rows (Chapter 6). Its lincheck, designed for exactly our segmented workload shape (batches of the same circuit with input-output chaining), is the direct competitor to our cross-chunk Twist-spanning arguments and deserves a diff-read once the PDF is reachable from an unblocked host.

4.5 The 2026 refinements: monomial basis, trace packing, tower arithmetic

Three 2026 papers refine the same engine at lower intensity. The monomial-basis (projective) sum-check of Dao, Biswas, Eagen, Milson, Papini, and Thaler (ePrint 2026/762) runs the protocol over the interpolating set $\{0, \text{infinity}\}^n$, where evaluation at infinity extracts monomial coefficients directly; near-drop-in, it measured about 10% end-to-end on two prime-field systems, eliminates all field subtractions when binding (free for us in characteristic two, where subtraction is XOR), simplifies eq and less-than interpolants, and - the part relevant to us - aligns naturally with WHIR-style low-rate commitments, which is the shape of our DeepWrap tail; our char-2 window is already the same object in disguise, and the projective view would simplify EqFactor and LT-factor evaluation in the verifier's structured public factors. Frigo and Shelat's sum-check bit packing (ePrint 2026/2770) packs m bits per $GF(2^m)$ element and verifies bit-linear claims via the GF(2)-linearity of the field trace with soundness error 2^{-m}, with zero-knowledge via a trace-zero characterization using only linear constraints - an explicitly cheaper alternative to Diamond-Posen ring-switching that interacts with our ZK-masking roadmap (as does Kochergin's zero-knowledge ring switching, ePrint 2026/2754, which bounds the mask cost at n-1 masks plus n-1 quadratic constraints). Liu and Zhang's tower sumcheck (ePrint 2025/594) reduces prover time for products of multilinears over a tower from $O(\text{kappa}^2\ N)$ to $O(\text{kappa}^{1.5}\ N)$ via a basis-switching mechanism - relevant to the bitsliced $GF(2^{128})$ kernels of magnetar-bitsum, where the wiring to the engine's factor interface is the remaining step. Finally, the time-space tradeoff paper (ePrint 2025/1473) proves $O(kN)/O(N^{1/k})$ optimal for non-adaptive provers - the theory license for streaming within chunks on our memory-bound host, where 2025/1117's 20x-class wins were measured precisely in the memory-limited regime.

4.6 Round-count and the two structural items

Two items close the chapter. The log-star-round IOPs of Athamnah, Ron-Zewi, and Rothblum (ePrint 2025/1269) - O(log* S) rounds with a linear prover and O(log* S) queries for a large class of Boolean circuits - matter to us only at the GPU milestone, where the alternation between IOP message computation and Fiat-Shamir hashing becomes the serialization bottleneck; it is a circuit-IOP reconstruction rather than a drop-in for Spartan R1CS, so it stays a Phase-5 research item exactly as the roadmap placed it. And the SuperSpartan 'next multilinear extension' technique (Setty, Thaler, Wahby, ePrint 2023/552) - the one BinarySpartan-list item this repository has not implemented (STATUS section 8) - remains PDF-gated from this sandbox; what can be said is that our cross-step machinery already implements a next-MLE shift argument for the PC chain (fetch(j+1) = TT[fetch(j), taken, halt]), so the residual exposure is applying the successor decomposition to witness columns - which is also the documented JALR extension path. Closing STATUS section 8 properly requires one PDF-retrieval session from an unblocked host; no implementation should start from the reconstruction alone.

Technique (ePrint)	Our layer	Their measured anchor	Projected effect here	Effort	Verdict
Eq-tensor optimization (2026/587 T3; 2024/1210)	engine rounds + both outers	2-3x Spartan-in-Jolt; 25%/10% e2e prime fields	1.3-1.6x wall-clock (projection)	S-M	Must - first
Constraint packing (2024/1038)	same sites	~128x eq precompute cut, char-2 native	bundled with above	S	Must - bundle

Technique (ePrint)	Our layer	Their measured anchor	Projected effect here	Effort	Verdict
Small-value + streaming (2026/587 T2; 2025/1473)	window + engine memory regime	>10x runtime+memory on Jolt Spartan; O(kN)/O(N^1/k) optimal	memory-bound 4-vCPU host is the winning regime	M	Must
Window extension to Shout rounds (2026/587)	window.rs + shout	1.7-2.2x on that sub-protocol	~5% e2e (projection)	M	Must
Univariate skip generalization (2024/108)	skip-eligible rounds	2^k-factor base-field shift	covered partly by window	S	Should
AG skip (2026/2494)	first-round path	2.36x skip round / 1.65x zerocheck at 2^32 in Flock	~10-15% e2e; kappa=128 caveat	M-L	Phase 5
Flock lincheck + circuit walker (2026/1329)	segments + wrap lincheck	enables dense-AND rows; batch-hash SOTA	wrap-circuit enabler	M	Must (wrap)
Monomial/projective basis (2026/762)	engine + verifier factors	~10% e2e prime fields; no subtractions	simplifies Eq/LT factors; aligns with DeepWrap	S-M	Should
Bit packing via trace (2026/2770); ZK ring switch (2026/2754)	packing + ZK layer	soundness 2^-m with linear constraints	alternative to ring-switch + cheaper ZK	M	Optional
Tower sumcheck (2025/594)	magnetar-bitsum kernels	O(kappa^1.5 N) vs O(kappa^2 N)	GPU kernel constant	M	Optional
Log-star rounds (2025/1269)	PIOP layer (future)	O(log* S) rounds, linear prover	GPU-phase serialization relief	L	Phase 5
SuperSpartan next-MLE (2023/552)	cross-step / witness columns	(PDF-gated)	close STATUS section 8; JALR path	S (read) / M (port)	Verify first

Table 5: Sum-check and PIOP techniques mapped to our layers (anchors are the papers' own measurements).

5. Commitments, Accumulation and Proof Size

This chapter is organized around one measured fact: at 2^{22}, roughly 84% of the paper's 596 KiB proof is the evaluation argument's correlated Merkle-path serialization, and roughly 16% is sum-check transcript machinery. Every lever below therefore either shrinks the openings, changes what has to be opened, or changes how many opening claims survive composition. The verification pass also settled the citation questions this chapter depends on: [BMNW24] - the bounded-depth spot-check accumulation our wrap.rs implements with its fourteen consistency queries - is 'Accumulation without Homomorphism' (Bunz, Mishra, Nguyen, Wang, ePrint 2024/474); ARC is ePrint 2024/1731; DeepFold is ePrint 2024/1595 (USENIX Security 2025); and the two papers that matter most for our open accumulation problem are WARP (ePrint 2025/753) and Khatam (ePrint 2024/1843).

5.1 Fast encoders: Bolt, SPIN, and the code families behind them

Our commit phase is only 5% of prover time on the CPU host, so encoder speed is a GPU-milestone concern - the roadmap's own arithmetic (commit dominates once everything else speeds up 10x) - but the code family also determines opening structure, so the choice cannot be deferred indefinitely. Bolt (Gurkan, Novakovic, Rothblum, ePrint 2026/310, CRYPTO 2026) is the current reference: a code-switching MLPCS over 'sketched codes' with commitment cost of roughly (3+eps)N field additions plus a Merkle hash over (1+eps)N field elements - against Blaze's 8N adds and 4N-size hash - measuring 3-4x faster commitment than Reed-Solomon schemes, about 20% faster than Brakedown with 5-15x smaller proofs, and instantiated over GF(2^{32}) at N = 2^{30}, proving the binary-field route. Two siblings refine the menu: SPIN (ePrint 2026/2317) gives rate-1/2 codes with O(N) encoding in both the ordinary and the **transposed** direction - the transposed encode is exactly the shape a sum-check-driven prover needs when it walks codewords backwards - with provable distance above 0.10, Lean-verified, and 1.63-2.29x prover throughput inside Flock at the cost of larger proofs; and the RAA-over-large-fields result (ePrint 2026/524) shows RAA codes reach relative distance 1/2 at n = 2^{20} over a 31-bit field - and GF(2^{128}) is a large field natively - worth 2.6x smaller proofs for RAA-based PCS. The migration cost is the honest caveat: our proximity argument is specific to the affine-substitution codeword's near-distance-1 structure, and swapping code families means re-deriving it - which Khatam (Section 5.3) makes possible but which is real work, so the encoder swap is scheduled with the GPU phase, not before.

5.2 The Reed-Solomon line and the FRI-Binius wrap alternative

The RS-based proximity line defines the size frontier our wrap competes with. STIR (2024/390) brought query complexity to O(log d + lambda log log d) - 114 KiB at degree 2^{26}, rate 1/4, 128-bit, against 211 KiB for tuned FRI; WHIR (2024/1586) is the current standard, with a measured PCS at degree 222 and 100 bits of 63 KiB proofs, 360 microseconds verification, and 1.2 s commit-plus-open; Basefold (2023/1705) gave the multilinear O($\log^2$ n) verifier that HyperPlonk compositions use; LiftWHIR (2026/1561) cuts the WHIR evaluation-phase prover 4.4-6.7x via DEEP sampling plus interleaving for 1.33-1.46x proof growth; ReedWeave (2026/2147) interleaves before folding, at 594.7 KiB for d = 2^{24} in the prover-optimized corner. For us the actionable piece is FRI-Binius (Diamond and Posen, ePrint 2024/504, EUROCRYPT 2026, nine revisions through May 2026): the ring-switching sum-check compiler we already implement, plus a characteristic-2 adaptation of Basefold foldable codes, yielding polylogarithmic proofs for bit-valued multilinears - and its zero-knowledge variant (2025/1015) ports Aurora-style masking to characteristic two, compatible with our landed zk.rs masking. The design alternative for the wrap's public tail is therefore: fold the low-rate wrap codeword FRI-style with index-shared queries across fold rounds (FRI-Binius shape) instead of BMNW spot-checks, with LiftWHIR's DEEP-plus-interleaving discipline keeping the discharge cheap - our discharge phase is 34% of prover time, so the eval-phase prover numbers are the ones to beat. The paper's own stance - provable soundness, no proximity-gap conjectures - is the constraint both alternatives must respect (Section 5.5).

5.3 The accumulation problem is solved in the literature

RECURSION.md's stated research follow-on - 'porting ARC's unbounded shape needs either an RS-compatible commitment or a proximity-gap argument for the affine-substitution code' - is, as of 2025-2026, solved on both halves. Khatam (Zeilberger, ePrint 2024/1843, CRYPTO 2026) proves the proximity gap for **multilinear folding over all linear codes**: if pi_L + r pi_R is close to a codeword of any linear code C over any finite field, then pi_L, pi_R, and pi_L + r pi_R agree with codewords on a common position set of relative size greater than $((1 - \Delta_C + \varepsilon)^{1/3} + \eta)n$, with failure probability below $3/(\varepsilon \eta |F|)$ - which over GF(2^{128}) is negligible by any accounting, and which is precisely the theorem our fold chain needs (our pcs.md already cites the TR25-169 version of this line). WARP (Bunz, Chiesa, Fenzi, Wang, ePrint 2025/753) then supplies the scheme: the first accumulation scheme with linear prover time and logarithmic verifier time, hash-based (ROM), plausibly post-quantum, and supporting **unbounded accumulation depth**, built on an interactive oracle reduction of proximity that works with any linear code over a sufficiently large field - introducing straight-line round-by-round knowledge soundness precisely so that codes without efficient error-tolerant decoding (our affine-substitution codewords qualify) can still accumulate. The engineering translation: replace wrap.rs's bounded BMNW spot-check consistency (14 queries, epsilon-loss per fold, depth ceiling at the code's distance margin) with a WARP-shaped IOR over our own code, with Khatam's gap carrying the fold-chain soundness; the tree-IVC depth ceiling disappears, and the GPU Phase-3 'Arc folds on the chunk tail' item becomes sound at unbounded k. FICS/FACS (ePrint 2025/737) supply the neighboring linear-prover machinery (an IOP of proximity for multilinear evaluation with O(lambda log log n + log n) queries; NP accumulation with O(lambda) oracle queries per step, supporting systematic LDPC and tensor codes). This is the single highest-leverage research import available to the repository.

5.4 Opening compression: Canopy and the batch-query kit

The most direct attack on the 84% byte share is Vector Commit-and-Reduce (Baweja, Mishra, Mopuri, Namboothiry, ePrint 2026/2790, November 2026, with a public implementation). VCR generalizes vector commitments so the opening algorithm is an interactive reduction, and its Canopy construction 'greatly reduces proof size and verifier time by reducing the vector opening claims to proximity claims about certain committed codewords' - exactly the q-queries-times-full-paths to few-proximity-claims compression our correlated openings need. Their measurements: for SNARKs based on the Bolt code, minus 50% proof size and 3x fewer verifier hashes at 5-15% prover overhead; for RS-based SNARKs, minus 20% proofs at 20% prover overhead; plus hash-based PCD with O(lambda sqrt(log N)) recursion overhead. Applied to our measured anchors - FastChunk tail 62.8 KiB, DeepWrap 27.3 KiB, wrapped artifact 36.5 KiB, composed projection 128 KiB - the Bolt-code-class result projects the composed artifact to roughly 80-105 KiB, and even the RS-class minus 20% lands at ~105 KiB. Three smaller tools complete the kit: UltraFold (2026/266) packs multiple digests per Merkle leaf for shallower trees and fewer path nodes (our magnetar-hash packing degree is a free dial to re-measure); the batch polynomial evaluation MA-protocol of 2026/448 turns any univariate PCS into one where opening m points costs a constant overhead over opening one (our ring-switch anchor openings and any multi-point discharge); and two-proofs-of-work grinding (2026/2326) buys k bits of query budget at 2^k prover work if the tree-IVC union bound ever runs tight - a shelf item with a real security-analysis obligation. DeepFold (2024/1595) contributes its arbitrary-length batch evaluation for FRI-based multilinear PCS - our MosaicLayout pads every component to aligned power-of-two subcubes, and the one-hot address components dominate committed bits, so non-power-of-two batch commitment cuts committed inventory and commit work (though not proof bytes).

5.5 Soundness repricing and the digest question

The paper's refusal to assume proximity-gap conjectures was validated by the 2025-2026 literature, and the repricing has parameter consequences. Crites and Stewart (ePrint 2025/2046) disproved all three up-to-capacity conjectures the deployed systems were pricing against: correlated agreement up to capacity (BCIKS), the mutual correlated agreement conjecture WHIR relied on, and DEEP-FRI list-decodability up to capacity - and gave Johnson-bounded replacements. Chai and Fan (ePrint 2026/858) then restored unconditional soundness above the Johnson bound for FRI/STIR/WHIR via threshold halving - epsilon_FRI bounded by $nR/|F| + (1 - Delta/2)^q$, unchanged protocol, recalibrated queries at roughly 2x cost, valid in characteristic two via additive folding, and Lean-verified. Two further flags bound our own dial tuning: Dao, Kominers, Thaler, and Zheng (ePrint 2026/2424) construct explicit counterexamples showing beyond-Johnson proximity gaps **fail over binary additive domains** at rate 1/4 - so our low-rate CompactWrap/DeepWrap profiles must be set by the binary-field bounds, not the RS intuition; and Fenzi and Sanso (ePrint 2025/2197) show small-field hash-based SNARGs are less sound than conjectured near the capacity regime - the generic attack depends on list size, again pinning the low-rate dials. On digests: the roadmap's 16-byte BLAKE3 idea (cut path bytes ~45%) fails the conservative collision budget - in the ROM, binding loss of an n-bit-digest Merkle commitment against a Q-query adversary carries a collision term around $Q^2/2^{n+1}$, so $Q = 2^{60}$ at a 120-bit target needs $n >= 240$ bits; 128-bit digests cap collision-type security near 64 bits. A refined protocol-specific argument (the adversary needs a colliding leaf at a position the transcript will query) might rescue 16 bytes, but it would be an unpublished bespoke proof carrying our 120-bit claim - a poor trade for bytes that Canopy compresses at measured minus 50%. Recommendation: keep 32-byte digests in every public artifact; spend the plumbing effort on opening compression.

5.6 The lattice fallback shelf

For completeness: the Horizon-3 'hybrid lattice wrap' fallback now has concrete inhabitants. Neo and SuperNeo (Nguyen and Setty, ePrint 2026/242) are the first folding schemes with all six of post-quantum security, pay-per-bit commitment costs, field-native sum-check over a small field, general (non-SIMD) constraint systems, small-field support, and low recursion overhead - via folding-friendly Ajtai/Module-SIS commitments; their own framing ('hash-based schemes such as Arc incur large verifier circuits') is our wrap problem stated from the other side. PikkuFold (2026/1809) improves the lattice route's per-step constants to 5.7 KB. LaBinius (2026/2103) instantiates the lattice PCS with a binary-tower evaluation field, and BitZ (2026/2141) gives hash-based commitments for multilinear polynomials over arbitrary rings through binary fields with cost independent of the ring and free bit-size checks. None of these changes the recommendation - adopting any would mean a second proving stack over Z_q - but SuperNeo is the documented fallback if the pure wrap misses its size projection, and BitZ is the tool if non-binary guest arithmetic ever needs committing without bit-blasting cost accounting.

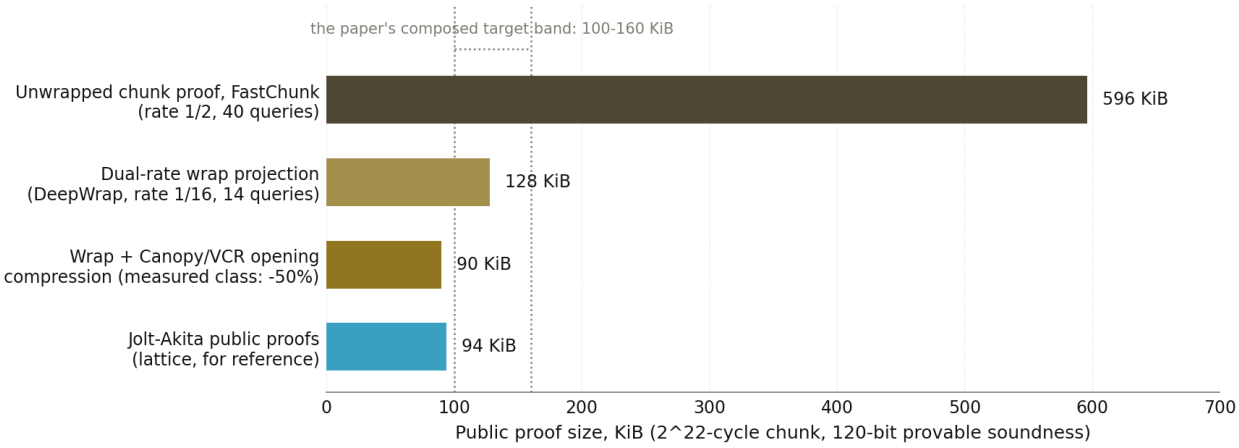


Figure 2: The proof-size path for the composed 2^22-cycle system. 596 KiB is the paper's unwrapped Table-6 figure; 128 KiB is this repository's measured dual-rate wrap projection; ~90 KiB applies Canopy-class opening compression (their measured -50% on Bolt-code systems; -20% would land ~105 KiB). Jolt-Akita's 94 KiB lattice proofs shown for reference.

Step	Size at 2^22	Status	Basis
Unwrapped chunk proof, FastChunk (rate 1/2, 40 queries)	596 KiB	Paper Table 6; repo tails measured (62.8 KiB eval tail)	Correlated Merkle paths dominate (~84%)
Dual-rate wrap (DeepWrap rate 1/16, 14 queries; BMNW spot-checks)	128 KiB composed projection	Implemented and measured on repo (wrapped artifact 36.5 KiB, 44% tail cut)	wrap.rs; bounded-depth accumulation
Canopy/VCR opening compression	~90-105 KiB	Not implemented	Their measurement: -50% on Bolt-code systems, -20% on RS, 3x fewer verifier hashes
WARP unbounded accumulation	holds at any depth	Not implemented (design import)	Any linear code over a large field; Khatam carries the fold chain
16-byte digests	rejected	Negative result	Collision budget needs ~240-bit digests at Q = 2^60 for 120-bit

Table 6: The proof-size path, step by step, with the status of each step.

6. The Verifier-Guest Wrap: Hash Arithmetization and Witness Compression

The wrap is the repository's Horizon-2 centerpiece: prove one implementation of our own verifier inside the guest, once, at a low rate, so the public artifact is the wrap's size rather than the chunk's. RECURSION.md already fixed the guest's workload for a 2^{22}-cycle chunk proof (~600 KiB, verifying natively in 10-12 ms): transcript hashing dominates - the SHA3-256 duplex absorbs roughly 9,600 Keccak-f1600 permutations; Merkle-path checks add about 3.1K BLAKE3 compressions (roughly 120 correlated paths of (n+k) digests); the GF(2^{128}) fold arithmetic is under 2^{24} bits. The design budget for the whole wrap witness is 2^{28}-2^{29} committed bits. The number that made this chapter necessary: at a naive ~132K witness bits per Keccak permutation, the transcript-hash component alone is 9,600 x 132K = 1.27 x 10^9 = $2^{30.2}$ bits - the budget is broken before the path checks even start. The 2025-2026 literature closes the gap, and Table 7 is the ladder.

Encoding	Bits per permutation	Total at 9,600 perms	vs the 2^28-2^29 budget	Source / status
Naive full arithmetization (our current estimate)	~132K	2^30.2	broken (4x over)	RECURSION.md estimate
Flock all-wires (s_0..s_24 + chi-outputs t_0..t_23)	78,400 (49 x 1600)	2^29.6	still over	Flock 2026/1329, measured
Flock only-ANDs (s_0, s_24, t_0..t_23)	41,600 (26 x 1600)	2^28.6	inside, no headroom	Flock 2026/1329, measured
Two-round degree-4 rows (commit every second round)	22,400 (14 x 1600)	2^27.7	inside, ~1.5-3x headroom	Arithmetic on Flock's encoding (derived)
Keccacheck GKR batch (commit in/out states only)	3,200 committed (2 x 1600)	2^24.9	inside, ~16-64x headroom	Keccacheck 2025/1764 (<4,000 constraints/perm at batch >= 512)

Table 7: The Keccak-f1600 witness-compression ladder for the wrap's transcript-hash component.

6.1 Why Keccak is cheap in characteristic two

The structural reason the ladder works is the char-2 arithmetic this stack is built on. In GF(2), XOR is field addition, so the linear steps of Keccak-f - theta, rho, pi, and the round constants of iota - cost nothing as constraints: they are absorbed into the linear layer of the R1CS/PLONK relation. The only nonlinear step is chi, five 2-input ANDs per lane position per round, and the only committed data that must survive is what the nonlinear step consumes and produces. Flock's two encodings make the trade explicit: committing all wires (49 x 1600 = 78,400 bits per permutation) keeps the constraint matrix sparse, while committing only the AND operands and outputs (26 x 1600 = 41,600) nearly halves the committed data at the price of roughly a hundredfold denser rows in the A_0 matrix - which is exactly what Flock's circuit walker (Chapter 4) exists to absorb: the lincheck's hot path is computed by a symbolic walk of the hash circuit in reverse topological order, never materializing the dense matrix. The two-round degree-4 refinement commits every second round's chi-operands (14 x 1600 = 22,400 bits) and runs the zerocheck at degree 4 across pairs of rounds, landing inside our 25-30K bits-per-permutation target; this last step is our arithmetic on top of their measured encoding, and the report marks it as derived rather than measured.

6.2 The GKR stretch: Keccacheck

The stretch variant replaces the flat batch-R1CS encoding with a data-parallel GKR sub-argument. Keccacheck (Kostrzewa, Klein, Adkins, Swirski, Zmuda, Reilabs, ePrint 2025/1764) batch-verifies Keccak permutations with sum-check plus GKR structure, reducing to multilinear-extension claims on the input and output states: fewer than 4,000 constraints per instance at batch sizes of 512 and above, more than 10x cheaper than the flat alternative, and competitive from batch 16 - with rollup storage proofs, light clients, and bridges stated as the motivation. Our transcript hash is precisely a batch of 9,600 permutations: commit only the per-permutation in/out states (2 x 1600 = 3,200 bits each, $2^{24.9}$ total), and discharge the 24 layered rounds as a data-parallel sum-check inside the Mikros discipline - our engine is already char-2 sum-check native, and Hashcaster's Frobenius multiopen and boolcheck tricks (over the (0,1,infinity) evaluation set) are the published prover-rate optimizations for exactly this shape. The honest costs: a new sub-protocol inside the wrap (medium-large effort), and the derivation of the per-round GKR wiring in char-2 form. The payoff is headroom - 16-64x under the budget - which is what buys room for 64-bit challenge fields or more queries later without re-deriving the wrap.

6.3 The BLAKE3 Merkle-path sub-circuit

The wrap's second workload - the correlated-path digest checks - is the item this report's parent task named explicitly, and it is the easier of the two. Flock's measurements give 82.1K BLAKE3 compressions per second per M4 Max core against 30.7K Keccak-f permutations - BLAKE3 is 2.67x cheaper in their stack - and scaling from their committed-bits ratio, a Flock-style BLAKE3 compression costs roughly 15-16K committed bits (derived, not measured). At 3.1K path-check compressions that is about $2^{25.5}$ bits, comfortably inside any rung of the ladder. The circuit shape follows Flock's IO glue: a batch-R1CS over GF(2) for the G-function chain (the 32-bit additions are carry chains, ~31 AND-shaped gates each, natural in binary fields), with a shift argument binding path-level chaining - the same successor-decomposition machinery our cross-step constraints already implement - replacing per-path permutation checks. Binius64's 2025 results (64-bit-word circuits with native XOR/AND/shift constraints and a 64-bit multiply constraint at 64x reduction versus bit-level; ring-switching closing the tiny-field proof-size gap, 0.844 MiB to 0.162 MiB at 2^{24}) are the engineering reference for word-level gates; with the caveat that Irreducible shut down in November 2025 and relicensed Binius64 MIT/Apache-2.0, so we import techniques, not a dependency.

22.4K
committed bits per Keccak permutation at the two-round degree-4 rung (derived from Flock's measured encoding)

<4,000
constraints per Keccak permutation in Keccacheck's GKR batch at batch >= 512 (measured)

~2^25.5
total committed bits for all 3.1K BLAKE3 path-check compressions (derived)

6.4 The minimal wrap circuit

Assembled, the minimal wrap relation is three flat sub-relations in the Mikros packed discipline, versioned with the verifier. First, transcript hashing: the 9,600 Keccak permutations as batch-R1CS over GF(2) per Flock, committing only s_0, s_24, and the chi-outputs (41,600 bits per permutation), with theta/rho/pi/iota absorbed as free linear layers; the two-round degree-4 refinement (22,400) where the density is absorbed by the circuit-walker lincheck; zerocheck accelerated by the univariate skip, packed challenges over GF(2^8) inside GF(2^{128}), and monomial-basis deferred reduction. The stretch variant swaps in the Keccacheck GKR batch at 3,200 committed bits per permutation. Second, Merkle paths: 3.1K BLAKE3 compressions as a second batch-R1CS with slot-aligned IO glue and the shift argument (~$2^{25.5}$ bits). Third, fold arithmetic: under 2^{24} bits of GF(2^{128}) relations for the round-sum checks and fold relations. All claims alpha-fold into one ring-switch evaluation argument - the Mosaic machinery we already run - proven once at DeepWrap. The wrap witness totals roughly 2^{26}-2^{27} bits flat, or ~$2^{25.5}$ with the GKR variant, against the 2^{28}-2^{29} budget and the naive $2^{30.2}$. At the BinarySpartan-paper rate of 287K Keccak permutations per second (CPU-side, including witness generation), the transcript-hash phase costs on the order of 33 ms of prover time per chunk - inside the 30-120 ms wrap budget that keeps the sequential-IVC overhead promise of 5-20% honest.

6.5 Keep the standard hashes

The alternative route - replacing SHA3/BLAKE3 inside the wrap and state layer with a SNARK-friendly hash over binary fields - was evaluated and rejected, and the rejection is now better-grounded than it was a year ago. The candidates exist: Poseidon2b (ePrint 2025/1893, binary-field Poseidon/Poseidon2 with the cryptanalysis redone for characteristic two, published in IACR Communications in Cryptology 2026) claims wins in proof size, proving time, and especially verification time; Vision Mark-32 (ePrint 2024/633) is optimized for binary towers with an FPGA pipeline at 250 MHz; Tip5 and Monolith are the Binius-lineage designs. The counter-arguments are three. The Flock and Keccacheck results above make the standard hashes cheap enough that the swap buys a bounded constant, not a regime change. The designs are young: the 2024/1900 collision attacks find conforming pairs for round-reduced Tip5, Tip4, Tip4', and Monolith through the lookup S-box. And the swap breaks the paper's section 1.1 posture - standard, heavily-scrutinized, non-algebraic hashes - exactly where QROM scrutiny concentrates: the wrap is the component whose soundness argument is a recursive composition. The industry trajectory agrees: Ethereum's post-quantum direction is hash-based (the XMSS-family multi-signature aggregation line, ePrint 2025/055, explicitly motivates BLAKE3-in-SNARK work), and the measured state-proving reference points (an Ethereum state proving service: 1,419 account reads at ~3 MiB of Merkle-Patricia data proven by Binius in 11-28 s CPU, or by SP1 in ~100M RISC-V cycles, ~5 min GPU, 1.2 MiB proof) confirm that keccak-in-circuit is the cost everyone pays - and that binary-field provers pay least. Verdict: standard hashes stay; the Flock/Keccacheck machinery is the compression.

7. Memory Checking and Lookup Arguments

Our BinaryTwist and BinaryShout implement the one-hot-addressing-plus-increments design of Twist and Shout (Setty, Thaler, Zhu, ePrint 2025/105, CRYPTO 2026) in characteristic two, with the linear-in-T provers, the Frobenius refinement of the Delta* one-hotness, and the address virtualization this repository has built and tamper-tested. Two findings from the survey bear directly on that layer. First, the paper's 2025 revision added corrections for one-hot checking in small-characteristic fields - our exact setting - so the first action is an audit: re-derive our one-hot checks against the revised text and port any delta, a small effort with a soundness payoff, and diff-read their d-parameter (bit-decomposition depth) tradeoffs against our twist.rs constants. Second, their measured advantages quantify what the design is worth where it wins: Twist commits 3.7x fewer nonzero values with 1.7x fewer field multiplications than Spice at 32 registers; Shout commits at least 8x fewer nonzeros with 3.4x fewer multiplications than Lasso on structured 2^{64} tables; inside Jolt the pair bought a 3x prover speedup and ~5x smaller proofs. The 2025/611 small-space line (Nair, Thaler, Zhu - the streaming shape our hierarchical root already implements) remains the memory-side reference for bounded-memory proving, with overhead 'potentially well below a factor of two' and no recursion.

7.1 The lookup landscape

The 2026 systematization of lookup arguments (Hafez, Anthoine, Campanelli, Fiore, ePrint 2025/1876) maps the four families - multiset-equality, LogUp-based, accumulator, and subvector-extraction - with prover-cost and recursion-compatibility guidance, and it is the map we should cite when choosing per table. Two concrete imports: Logup* (Soukhanov, ePrint 2025/946) is a faster LogUp for small-table indexed lookups - a direct fit for the Mikros interpreter's opcode tables at small effort; and the almost-linear-time permutation check of Bunz, J. Chen, DeStefano, and B. Chen (ePrint 2025/1850) gives BiPerm (O(n) field ops, PCS-restrictive) and MulPerm (n times near-log-squared, any PCS) with soundness polylog(n)/|F| - fixing the small-field soundness gap that blocks permutation checks over tiny fields, and the drop-in if BinarySpartan ever needs a permutation check over GF(2^k). The systems cross-check: SP1's global memory argument is LogUp-GKR over a fractional sum-check with Jagged commitments, Ziren runs one LogUp argument over all buses per shard (42% of its GPU kernel time), and OpenVM 2.0 pairs Zerocheck with LogUp-GKR - the field has converged on bus-style multiset arguments for unstructured tables, while our one-hot design keeps the committed-value advantage for the structured register/RAM ports where the addresses are dense. The division of labor, confirmed by both sides' measurements: keep Twist for the ports and RAM; adopt LogUp* for the small unstructured tables; leave the grand-product permutation machinery to Mikros's packed discipline where it already cross-validates.

Item	Action	Anchor	Effort
BinaryTwist one-hot checks (char-2)	Audit against Twist-and-Shout 2025 revision; port small-characteristic corrections; diff d-parameter	2025/105 revision; 3.7x/1.7x vs Spice measured	S-M
Small-space memory proving	Already adopted (hierarchical root); extend to per-segment Twist/Shout provers (STATUS section 5)	2025/611: below 2x overhead, no recursion	M (planned)
Mikros interp tables	Logup* small-table indexed lookups	2025/946	S
Permutation checks over GF(2^k)	MulPerm/BiPerm if needed	2025/1850: polylog(n)/|F| soundness	M
Lookup family map	Adopt 2025/1876 SoK as the choosing rule per table	-	S (citation)

Table 8: Memory and lookup decisions.

8. Folding, IVC, PCD and the Stateless Layer

The negative structural result first, because it saves effort: classical folding does not apply to this stack, and no 2025-2026 paper changed that. Folding schemes homomorphically combine witness-commitment pairs, which requires an additively homomorphic commitment - elliptic-curve Pedersen commitments in the Nova/CycleFold/ProtoStar lineage (ePrint 2023/1192, 2023/620, 2022/1758), pairings in Mira (2024/2025, whose Groth16 aggregation gains 5.8x prover and 9.7x memory are real but pairing-bound). Merkle-tree and Ligero-family commitments are not homomorphic, so the three viable routes for a hash-based stack remain: accumulation (hash-based - ARC, WARP, the subject of Section 5.3), lattice folding (Ajtai commitments are additively homomorphic over Z_q - Neo/SuperNeo 2026/242 and PikkuFold 2026/1809 are the current best, at the cost of a second proving stack), or recursive wrapping (our design - prove the verifier). The survey's confirmation is useful in the other direction too: SuperNeo's authors state plainly that 'hash-based schemes (e.g., Arc) incur large verifier circuits' - which is our wrap problem stated from the lattice side, and evidence that no one has a free lunch here.

8.1 PCD plumbing and distributed Fiat-Shamir

The PCD layer around accumulation matured without surprises: holography-based accumulation for PCD (2026/538), zero-knowledge PCD from accumulation schemes (2026/289), and the simulation-extractability of PCD (2025/2037) supply the composability statements a production tree-IVC needs; VCR's O(lambda sqrt(log N)) PCD recursion overhead (2026/2790) is the current published bound our tree should be measured against. One negative finding matters for our RECURSION.md design: no published paper under the name 'distributed Fiat-Shamir' was found on ePrint or arXiv - the coordinator-star distributed Fiat-Shamir that keeps our tree hash-only is, as far as this survey could establish, ahead of the published literature, which is a reason to document it carefully and, if the GPU milestone makes it load-bearing, to consider writing it up. The order-free LtHash aggregation inside ZisK (Chapter 3) is the systems-grade cousin of the same idea and the pattern our segment workers should adopt when they scale past a single machine.

8.2 The stateless layer and the 2^22-chunk validator

On vector commitments for state, the survey confirmed that nothing in 2024-2026 displaces plain hash-based Merkle trees for our threat model: the interesting results target other settings - efficient updates (Tas and Boneh, 2023/1830), query-throughput incremental authenticated dictionaries (Bender, Kuszmaul, Kuszmaul, 2026/2098), post-quantum VCs with efficient insertions and deletions (Pathak and Ruj, 2026/2592), and reconfigurable distributed VCs (BORG, 2026/1552). Ethereum's own roadmap page (updated October 2026) still marks weak statelessness as requiring Verkle trees plus PBS and 'probably a few years away,' with zkEVM-type L1 verification named as complementary - and the post-quantum direction (the hash-multisig line, 2025/055) is hash-based. The composition the parent task named - the 2^{22}-chunk validator - therefore assembles entirely from pieces this report has already priced: 2^{22}-cycle chunks at the FastChunk profile (the paper's Table-6 top row, our target operating point), the tree-IVC with shared commitments and boundary chaining (implemented), WARP accumulation replacing the bounded BMNW layer (Section 5.3), per-chip trace splitting with LtHash order-free aggregation for the worker fleet (ZisK pattern), the verifier-guest wrap at DeepWrap for the public artifact (Chapter 6), and Canopy compression on the openings (Section 5.4) - yielding a validator that proves in bounded per-machine memory, composes to arbitrary depth at 120-bit provable soundness, and publishes 90-105 KiB per chunk. The oracle-aided IVC workstream (ePrint 2026/2824, the updatable-hash forest and signed-oracle layer already planned in depth in the repository's W1-W12 plan) composes on top of this as the updatable-state extension; its interactions with WARP accumulation are limited to sharing the transcript discipline, and it is not re-researched here.

9. GPU and Hardware Acceleration

The GPU milestone (the repository's stated goal of at least 150 million proven cycles per second on the least GPUs) is where the 2026 external evidence is densest and most encouraging. Table 9 collects the frontier; the sections below translate it into the four decisions the port actually needs: the pipeline shape, the kernel discipline, the toolchain, and the memory-bandwidth arithmetic. One toolchain correction from the verification pass: the NVlabs cuTile project (now NVIDIA/cuda-rust, with cutile-rs on the production track) ships only AI kernels - attention, GEMMs, norms - through the CUDA Tile IR; **no public ZK kernel library exists on it**, so our feature-flagged bitsliced sum-check core and cuTile experiments must be re-evaluated against plain CUDA/Rust with cuPQC as the fusion layer, or we write the tile kernels ourselves on top of the framework.

System / paper	Hardware	Throughput	Notes
Airbender (ZKsync, 06/2025)	1x H100	21.8M c/s base; 8.5M e2e w/ recursion	Public reproducible harness; SP1 Turbo 3.45M and RISC Zero 1.1M in the same harness
Airbender Prime (10/2026)	1x RTX PRO 6000 Blackwell	99% of 7,200 mainnet blocks <10 s incl. witness	GKR sum-check + WHIR + BLAKE2s; ETHproofs leaderboard
Ziren (ZKM, 2026/2330)	1x RTX 5090 / 8x GPU	12.0M / 75.8M c/s	Kernel anatomy published: LogUp-GKR 42%, WHIR commit+open 19%, zerocheck 23.6%, jagged 8%

System / paper	Hardware	Throughput	Notes
OpenVM 2.0 SWIRL (01/2026)	1x RTX 5090 / 16x	11.4M / 139M c/s	p99 11.8 s over 1,000 mainnet blocks; AOT tracer at 3.8 GHz
SP1 Hypercube (11/2025)	16x RTX 5090	99.7% of blocks <12 s	Cluster; 12.5x GPU-efficiency gain in six months
Jolt, Apple Metal (09/2026)	M4 Max iGPU	>10M c/s	Lattice (Akita) commitments; consumer-GPU existence proof
Chimera (2026/1388)	GPU (BN254 Spartan, N=2^17)	sum-check region 672.8 -> 84.2 ms (7.99x)	The residency/hybrid-rounds discipline, measured
TensorZKP (2026/1250)	GPU tensor cores	(HyperPlonk+Brakedown at 2^25)	Sum-check and Brakedown encode as matrix multiplication on tensor cores
SNAIL (2025/728)	1x RTX 4090	prover at 21-28% of native, depth-24-32 circuits	2.6-billion-gate trace at d=32 in ~40 ms; the 'proving as fast as computing' bound

Table 9: The 2026 GPU proving frontier (their own reported numbers).

9.1 The pipeline pattern, made concrete

Airbender's architecture is the template because it publishes both the shape and the numbers. The CPU runs RISC-V simulation and tracing only; a GPU gpu_witness_eval_generator that mirrors the CPU implementation expands circuit-specific witnesses on-device; five proving stages run GPU-resident with an asynchronous allocator, pipelining that overlaps host-device transfers with compute, and CUDA graphs for launch stability; late small rounds switch back to the CPU - the hybrid discipline Chimera (ePrint 2026/1388) measured at 7.99x on the sum-check region alone by keeping polynomial data on-device across rounds and shipping only transcript values to the host. Ziren's published kernel anatomy then tells us where the effort pays first: bus arguments (LogUp-GKR at 42% of kernel time) and commitment hashing (WHIR commit+open at 19%) dominate, zerocheck is second (23.6% across compiled and interpreted paths), and the jagged reduction is 8% - so for our stack the port order is: the Merkle/encode kernels and the bus/Twist rounds first (they are also our two largest CPU phases, 32% and the commit side of the 34% discharge), the Spartan outers second, everything else third. PipeSC's ASIC study (ePrint 2026/691, up to 5.02x over a GPU implementation via pipelining and dependency-aware scheduling) is not a hardware plan for us, but its conclusion is the design constraint: sum-check is bandwidth- and dependency-bound, not FLOP-bound - which is why residency, fusion, and tiling (the roadmap's Phase 2) are where the multipliers live.

9.2 The tensor-core spike

One research result deserves a line of its own in the plan: TensorZKP (ePrint 2026/1250) reformulates sum-check and Brakedown/Spielman encoding as matrix multiplication and maps them onto GPU tensor cores with an async warp-specialized pipeline, instantiated end-to-end as HyperPlonk plus Brakedown at 2^{25}. The fit to our stack is unusually direct: our bitsliced GF(2^{128}) multiplication is a GF(2)-bilinear map - a Boolean matrix multiplication - and our MosaicPCS is Brakedown-family, so both the dominant sum-check rounds and the encode phase are natural tensor-core workloads. The roadmap's Phase-5 'tensor-core spike' should be promoted to a Phase-2/3 experiment once the straight port is stable; the expected gain class (2-5x on the matmul-shaped kernels) is the same class MORPH (arXiv 2604.17808, MSM and NTT remapped onto AI-ASIC MXUs) projects for AI-accelerator proving - worth a probe, not yet a dependency.

9.3 Toolchain reality

The September 2026 Rust-CUDA landscape, as verified: cutile-rs is on the production track and cudarc on the stable track, but the tile-programming frameworks ship only AI kernels - our ZK kernels would be written on top, not pulled from a library. NVIDIA's cuPQC (September 2026) is the consequential release for us: a CUDA SDK that exposes hashes, Merkle tree operations, and field arithmetic as device functions, enabling kernel fusion and vendor-maintained per-architecture optimization - SP1 is already integrating it, and it is exactly the primitive layer a BLAKE3-plus-encode-plus-sum-check fusion needs. ICICLE remains the prime-field reference implementation (less useful for GF(2^{128})). And a scheduling note for the plan: no public ZPrize edition exists after the May 2024 winners post, so the prize literature is closed as a source - the systems papers above are the evidence base now.

9.4 The bandwidth roofline and the governor

The roadmap's own arithmetic survives the survey intact and sharpened. The binding kernel - tower-field sum-check over the ~6.4 GiB of tables at 2^{22} - is a streaming workload: 30-60 GiB of effective traffic per chunk against an H100's 3.35 TB/s or a B300's 8 TB/s HBM3e gives a 4-18 ms roofline floor per chunk, so 150M cycles/s on one GPU is a bandwidth calculation, not a hope - the multiplier is memory-side efficiency (fusion, tiling, residency), which is why Chimera's 7.99x and Ziren's kernel shares matter more than peak FLOPs. The governor is the tracer: our host-side witness generation runs at roughly 450K cycles/s inside today's 4% phase, which caps any GPU pipeline that feeds it serially; the industry answer is the AOT tracer (OpenVM's 3.8 GHz single-pass compilation, 25x over interpretation) plus GPU witness expansion, with double-buffered upload - the roadmap's Phase-4 risk with a measured precedent. The phase plan (single-GPU port at >=2x the CPU baseline and >=15M c/s on one H100; fusion and tiling to >=40M; sharding and accumulation to >=100M on two GPUs; the tracer to end-to-end >=150M) is calibrated against Table 9's single-GPU cluster at 11-22M and the cluster points at 75.8-139M - each phase exit criterion sits inside ground the frontier already occupies, which is the strongest statement a research plan can make.

10. The Integrated Implementation Roadmap

This chapter assembles the survey into the phased plan the repository's ROADMAP.md grid already sketches, with every phase carrying: the items (each traceable to a chapter and an ePrint number), the multiplicative arithmetic on our measured baseline, and an acceptance criterion that is measured, not modeled. Two disciplines carry over unchanged from the repository's release history: every optimization must leave the verifier untouched and the round messages byte-identical (the property that has kept the tamper suites meaningful through the 0.9.x-0.12.x ladder), and every claim lands in BENCHMARKS.md with its hardware stated. Nothing below is implemented; the plan is the deliverable.

Phase 0 - Engine algebra (weeks; CPU-side, no protocol changes)

- Item 1+2: the eq-tensor fold and binary-tower constraint packing in the engine's round path and both Spartan outers (Ch. 4; 2026/587 T3, 2024/1038). Acceptance: MAGNETAR_TIME shows the Spartan phase down by the projected 1.6-2x; round messages byte-identical.
- Item 3: the window pass generalized to the Shout high-degree sum-check (Ch. 4; 2026/587 T2). Acceptance: the bus/Shout phases measurable in the anatomy, ~1.7-2.2x on that sub-protocol.
- Item 4: the Twist-and-Shout 2025-revision audit of the char-2 one-hot checks (Ch. 7; 2025/105). Acceptance: a written diff against the revised paper; any correction ported and tamper-tested.
- Item (bundled): monomial-basis / projective round identities where they simplify EqFactor and LT factors (Ch. 4; 2026/762).

Phase 1 - Arithmetization restructure (a quarter; the algorithmic-gap closer)

- Item 5a: the committed-bits-per-cycle cost census (Ziren methodology, Ch. 3) - committed cells, bus pairs, and field multiplications per RV64I opcode class, published in docs/BENCHMARKS.md. This gates 5b.
- Item 5b: per-opcode chips with instruction frames replacing the monolithic 512-row cycle relation, where the census says it pays (Ch. 3; Ziren 2026/2330).
- Item 6: jagged mixed-height commitment + LSB-paired folding in the Mosaic layout and the router join (Ch. 3; SP1 Jagged PCS, Ziren 10.3; -28% measured there).
- Item (parallel): per-chip trace splitting with continuation buses and LtHash order-free aggregation for the segment fleet (Ch. 3; ZisK).
- Acceptance: the erc20-mixed-block ladder at 2^16-2^20 re-measured; the per-core rate at or above ~10K cycles/s/vCPU; committed bits per cycle down by the census-predicted factor.

Phase 2 - The wrap and the proof-size kit (a quarter; Horizon 2 completion)

- Item 7: the verifier-guest wrap circuit - Keccak at only-ANDs 41.6K bits/perm (flat) or 22.4K (two-round degree-4), BLAKE3 path sub-circuit at ~15-16K bits/compression, fold arithmetic, one ring-switch evaluation at DeepWrap (Ch. 6; Flock 2026/1329).
- Item 8 (stretch): the Keccacheck GKR batch sub-argument at 3,200 committed bits/perm (Ch. 6; 2025/1764).
- Item 9: Canopy/VCR opening compression wired into magnetar-pcs::eval and the wrap (Ch. 5; 2026/2790).

- Item 10: the WARP-shaped unbounded accumulation layer over the affine-substitution code, with Khatam carrying the fold chain (Ch. 5; 2025/753 + 2024/1843); evaluate the FRI-Binius fold alternative on measured bytes before committing (Ch. 5; 2024/504 + LiftWHIR 2026/1561).
- Acceptance: the composed artifact measured at <=128 KiB pre-compression and ~90-105 KiB post; the wrap witness inventory printed by proof-stats inside 2^28-2^29; the tree-IVC depth ceiling removed in a soundness note.

Phase 3 - The GPU port (the milestone)

- The Airbender pattern: CPU trace + GPU witness expansion + GPU-resident proving with transfer-compute overlap; the Chimera hybrid-rounds discipline; cuPQC as the fusion layer; plain CUDA/Rust kernels on cutile-rs or cudarc, written by us (Ch. 9).
- Kernel order from the Ziren anatomy: Merkle/encode and bus/Twist rounds first, Spartan outers second; the tensor-core spike for the bitsliced multiply and the encode once stable (Ch. 9; TensorZKP 2026/1250).
- The AOT tracer when the pipeline exceeds ~1M c/s (Ch. 3/9; OpenVM 3.8 GHz pattern); sharding and the accumulation fleet at Phase-3 scale (ZisK splitting + LtHash + WARP).
- Acceptance: the ROADMAP grid's exits - >=15M c/s on one H100, >=40M with fusion/tiling, >=100M on two GPUs, end-to-end >=150M with the tracer - each inside the ground Table 9 occupies.

10.1 The arithmetic, stated

On the current 4-vCPU host, the measured baseline is 15.4-17.2K cycles/s at 2^{16} (about 4.3K per vCPU). Phase 0 applies the eq-tensor and packing gains to the sum-check-bound 63% (Spartan 29% + discharge 34%) at the papers' measured 1.6-2x on those sub-protocols, plus ~5% from the Shout window and 1.1-1.5x on the 32% Twist phase from the revision deltas - a multiplicative 1.4-1.7x, landing at 22-29K. Phase 1's restructure is the least certain multiplier: if the census confirms the Ziren-class committed-area ratios, 1.5-2x is defensible, landing at 35-58K - a per-core rate of 9-15K against the paper's hardware-normalized ~28K per core, with the residual 2-3x in per-kernel constants this repository's release ladder has historically closed at 1.5-2x per pass (the 0.11.x kernel releases each measured 1.5-1.9x). On GPU: 96% of prover time is GPU-able; at kernel gains of 15-30x the wall-clock multiplier is 7-14x on the current arithmetization and scales with the Phase-1 gains - 0.4-0.7M cycles/s per datacenter GPU pre-restructure and 2-6M after, with cluster scaling at the Ziren-measured 3.1-3.3x per four GPUs putting the 150M program target at the 32-64-GPU class the frontier occupies. Every number in this paragraph is a projection and is labeled as one; the anchors and their sources are Chapters 3-9.

Stage	Multiplier	Resulting rate	Basis
Baseline (measured)	-	15.4-17.2K c/s (4 vCPU)	BENCHMARKS.md, erc20-mixed-block 2^16
+ Phase 0 (eq-tensor, packing, window, Twist audit)	x1.4-1.7	22-29K	2026/587 + 2024/1038 measured 1.6-2x on the sub-protocols; 2025/105 deltas
+ Phase 1 (census, chips, jagged, splitting)	x1.5-2 (medium confidence)	35-58K	Ziren committed-area ratios; -28% jagged; gated on our census
+ kernel-constant passes (history)	x1.5-2 per pass	toward the ~112K hardware-normalized paper point	0.11.x release history: 1.5-1.9x per pass
GPU Phase 3 (96% of time at 15-30x kernel gain)	x7-14 pre-restructure	0.4-0.7M per GPU; 2-6M after Phase 1	Airbender/Chimera/TensorZKP classes; Ziren/OpenVM per-GPU anchors 11-22M
Cluster (3.1-3.3x per 4 GPUs)	-	150M at the 32-64-GPU class	Ziren 8-GPU 75.8M; OpenVM 16-GPU 139M

Table 10: The multiplicative projection (all projections; anchors cited in the chapters).

10.2 What changes in the repository's documents

Concretely, when implementation is approved: ROADMAP.md's Horizon-1 'small-domain sum-check arithmetic' row splits into the eq-tensor/packing and window-extension items with their measured anchors; the Horizon-2 'verifier-guest circuit' row gains the Table 7 ladder and the Keccacheck stretch; the Horizon-3 lattice-wrap row cites Neo/SuperNeo and BitZ as the named fallbacks; the GPU grid's Phase-1 toolchain row replaces 'cuTile kernels' with 'cuPQC fusion + our kernels on cutile-rs/cudarc' and its Phase-5 research row keeps AG-sumcheck and log-star with corrected citations (2026/2494, 2025/1269); STATUS.md section 8 (SuperSpartan next-MLE) stays open until the PDF retrieval; and the BENCHMARKS.md paper-comparison section gains the hardware-normalized per-core column this report introduced. No document claims a projected number as a measurement.

11. Risk Register and Negative Results

A survey earns its keep by what it refuses. This chapter collects the corrections, the unverified claims we will not act on, the soundness constraints that bind the parameter dials, and the deliberate rejections. The general method note: four research agents made roughly 175 primary-source opens across this survey; ePrint PDF endpoints were Cloudflare-gated from the sandbox, so full-text claims rest on abstract pages, the locally uploaded papers (Mikros 2026/2646, the oracle-aided IVC 2026/2824), the Flock arXiv PDF, official benchmark pages, and repository documentation; every claim below is marked by where it came from, and items that could not be verified are named as such rather than dropped silently.

Claim (as it stood)	Verdict	What the sources say
FRI-Binius is ePrint 2025/...	Corrected	It is 2024/504 (Diamond and Posen, EUROCRYPT 2026, rev. through 05/2026); the ZK variant is 2025/1015

Claim (as it stood)	Verdict	What the sources say
AG-sumcheck gives 2.36x	Qualified	2.36x is the skip round inside Flock at 2^32; the whole-zerocheck figure is 1.65x (2026/2494)
Airbender is the GPU port of Jolt	Corrected	It is ZKsync/Matter Labs' own STARK-lineage prover (zksync.io, matter-labs/zksync-airbender)
ZisK's tracer runs at ~1.5 GHz	Unverified	Not in the whitepaper, docs, or releases we opened; the verified fast-tracer story is OpenVM's 3.8 GHz AOT
'Deep', ePrint 2025/2691	Does not exist	The URL 404s; no such paper in the archive sweep; dropped
'o1j', 'MobileProofs', 'Bicycle', 'Berlin', 'Flashgate', 'Snakes' (as folding/lookup schemes)	Not found	No such papers located on ePrint/arXiv; Bicycle is plausibly a garbled 'Bird' (2025/2249); dropped from leads
Binius CRYPTO'25-titled paper on ePrint	Not findable	The canonical citations are 2023/1784 and 2024/504; only LaBinius and FRIVail match 'Binius' in the archive
ZPrize 2024/2025 results	Not found	No public edition after the May 2024 winners post
cuTile ships ZK kernels	Corrected	NVIDIA/cuda-rust ships AI kernels only; we write the ZK kernels on top

Table 11: Corrections and unverified items from the leads list.

11.1 Soundness constraints that bind

- Capacity conjectures are dead: correlated-agreement-up-to-capacity, the WHIR mutual conjecture, and DEEP-FRI list-decodability up to capacity are all disproved (2025/2046); parameters must be set by Johnson-bounded replacements or the threshold-halving repair at ~2x queries (2026/858, Lean-verified, valid in char 2).
- Binary additive domains are special: beyond-Johnson proximity gaps fail over them at rate 1/4 with explicit counterexamples (2026/2424) - our low-rate CompactWrap/DeepWrap dials must be derived from the binary-field bounds, not RS intuition.
- Small-field hash-based SNARGs near the capacity regime are less sound than conjectured (2025/2197) - same conclusion for the low-rate dials.
- 16-byte digests are rejected: the conservative ROM collision budget at Q = 2^60 needs ~240-bit digests for a 120-bit claim; 128-bit digests cap near 64 bits of collision security (Section 5.5).
- The AG code's product distance (1 - 2^-119) fits the 120-bit budget; the kappa = 128 packing caveat must be resolved before any implementation (Section 4.3).
- Flock's Keccak encodings and Keccacheck's constraint counts are the measured anchors; our 22.4K bits/perm and 15-16K bits/compression figures are derived from them and must be re-measured on our stack before they enter any budget claim.

11.2 Deliberate rejections

- No lattice fork: Neo/SuperNeo (2026/242) and PikkuFold (2026/1809) are excellent papers that would require a second proving stack over Z_q; the hash-only posture is the product. They stay on the Horizon-3 fallback shelf with LaBinius (2026/2103) and BitZ (2026/2141).

- No SNARK-friendly hash swap: Poseidon2b (2025/1893) and Vision Mark-32 (2024/633) are documented alternatives; the Flock/Keccacheck results plus the 2024/1900 round-reduced collisions on Tip5/Monolith keep SHA3 and BLAKE3 in place (Section 6.5).
- No digest truncation (Section 5.5); the same bytes are better spent on Canopy compression with published measurements.
- No classical folding integration (Chapter 8) - structurally impossible without homomorphic commitments.
- No implementation from this survey before explicit approval: the parent instruction gates every item above on a separate go signal.

12. Consolidated Bibliography

Entries marked [ePrint] were verified on their IACR ePrint pages during this survey (November 2026); entries marked [web] are official project pages, blogs, or repositories opened by the research agents; entries marked [repo] are this repository's own documents. Grouping follows the chapters.

Cluster A - zkVM systems and hardware

Srinath Setty. Jolt: zkVMs through the looking glass. IACR ePrint 2023/1217. [ePrint] - the lookup-singularity zkVM this stack's CPU layer is compared against.

Srinath Setty. Lasso from the lookup singularity. IACR ePrint 2023/1216. [ePrint] - the lookup foundation under Jolt.

Srinath Setty and Justin Thaler. Twist and Shout: Faster memory checking arguments via one-hot addressing and increments. IACR ePrint 2025/105, CRYPTO 2026. [ePrint] - our BinaryTwist/BinaryShout lineage; the 2025 revision carries small-characteristic corrections.

Jolt team (a16z crypto). Jolt: an update (11/2024); Jolt's 6x speedup (08/2025); 64-bit proving in Jolt (10/2025); Lattice SNARKs: Jolt goes post-quantum and faster (09/2026). a16zcrypto.com posts. [web] - the release cadence and measured numbers.

Akita: A high-performance lattice-based polynomial commitment scheme. IACR ePrint 2026/1983. [ePrint] - Module-SIS PCS with commitment time proportional to nonzero entries.

ZKsync/Matter Labs. Introducing Airbender (06/2025); Airbender product page; Introducing Airbender Prime (10/2026); github.com/matter-labs/zksync-airbender (incl. gpu.md, end_to_end.md). [web] - the GPU pipeline template and the 21.8M/8.5M per-H100 numbers.

Succinct. SP1 Hypercube is now live on mainnet (02/2026); Introducing Flock (06/2026); SP1 x NVIDIA cuPQC (09/2026); The 2025 recap (12/2025). blog.succinct.xyz. [web] - Jagged PCS, the 16x5090 cluster result, cuPQC.

Polygon/ZisK. Zisk whitepaper and documentation (0xpolygonhermez.github.io/zisk-docs); releases v0.18-v1.4.0-alpha. [web] - per-chip trace splitting, continuation buses, LtHash aggregation, the OP-Mainnet census.

Duan, Yin, Hou, Pham, Zhu, Guo. Ziren: scalable RISC-V (MIPS32) proving on GPUs. IACR ePrint 2026/2330. [ePrint] - per-opcode chips, jagged sum-check, LSB pairing, the kernel anatomy, Lean theorems.

Axiom. OpenVM blog and docs (openvm.dev); OpenVM 2.0 Powered by SWIRL (01/2026); github.com/openvm-org/openvm. [web] - the SWIRL stack, 11.4M/139M numbers, the 3.8 GHz AOT tracer.

Kattis, Klatt, Quirk, Allen (Lita). A DSL for Valida zkVM programs. IACR ePrint 2025/1110. [ePrint] - the memory-direct STARK design.

LayerZero Labs and Succinct. vApps: native-performance zero-knowledge applications. arXiv 2504.14809. [web] - the 197x cycles and precompile acceleration results.

Farheen and Tsoutsos. Chimera: a hybrid GPU backend for sumcheck acceleration. IACR ePrint 2026/1388. [ePrint] - GPU residency and hybrid CPU/GPU rounds; 7.99x on the sum-check region.

Lu, Zhang, Guo, Liu, Qu, Wang. TensorZKP: privacy-preserving zero-knowledge proof on GPU tensor cores. IACR ePrint 2026/1250. [ePrint] - sum-check and Brakedown encoding as tensor-core matmuls.

PipeSC: a pipelined hardware accelerator for sum-check. IACR ePrint 2026/691. [ePrint] - the bandwidth/dependency-bound conclusion.

Tong et al. MORPH: zkML MSM/NTT on AI ASICs. arXiv 2604.17808. [web] - the AI-accelerator direction.

SNAIL: efficient arguments for layered Boolean circuits. IACR ePrint 2025/728. [ePrint] - prover within 21-28% of native; the proving-vs-computing bound.

NVIDIA. cuda-rust / cutile-rs repositories; cuPQC CUDA SDK announcement (09/2026). [web] - the toolchain reality check.

Ingonyama. ICICLE v3. github.com/ingonyama-zk/icicle. [web] - the prime-field GPU reference.

Cluster B - sum-check and binary-field PIOPs

Bagad, Dao, Domb, Thaler. Speeding up sum-check proving. IACR ePrint 2025/1117 (merging 2024/1046 and 2024/1210). [ePrint] - small-value rounds and the eq(r,x) dot p(x) optimization; 2-3x on Spartan-in-Jolt.

Dao, DeStefano, Bagad, Domb, Thaler. Speeding up sum-check proving (extended version). IACR ePrint 2026/587. [ePrint] - many-factor products, the streaming prover, the eq-tensor technique; >10x on Jolt's Spartan sub-protocol.

Dao and Thaler. Constraint packing. IACR ePrint 2024/1038. [ePrint] - pinned subfield challenges; the ~log|F| eq precompute cut over binary towers.

Angus Gruen. Some improvements for the PIOP for zerocheck. IACR ePrint 2024/108. [ePrint] - the univariate skip and its cost formula.

Dalal, Bunz, Hemo, Rothblum. Speeding up sumcheck using algebraic geometry. IACR ePrint 2026/2494. [ePrint] - the genus-95 multiplication code; 2.36x skip round / 1.65x zerocheck in Flock at 2^32.

Bunz, Rothblum, Wang. Flock: fast proving for batch boolean computations. IACR ePrint 2026/1329; arXiv 2607.27491; github.com/succinctlabs/flock. [ePrint] - the batch-hash SNARK of record; the circuit walker; the Keccak encodings.

Dao, Biswas, Eagen, Milson, Papini, Thaler. The sum-check protocol over the monomial basis, and other optimizations. IACR ePrint 2026/762. [ePrint] - projective sum-check over {0, infinity}^n.

Frigo and Shelat. Sumcheck bit packing. IACR ePrint 2026/2770. [ePrint] - trace-linearity packing as the ring-switching alternative.

Kochergin. Zero-knowledge ring switching. IACR ePrint 2026/2754. [ePrint] - the n-1 mask bound.

Liu and Zhang. Efficient SNARKs for Boolean circuits via sumcheck over tower fields. IACR ePrint 2025/594. [ePrint] - O(kappa^1.5 N) tower sum-check.

Baweja, Chiesa, Fedele, Fenzi, Mishra, Mopuri, Zitek-Estrada. Time-space trade-offs for sumcheck. IACR ePrint 2025/1473. [ePrint] - O(kN)/O(N^(1/k)), optimal for non-adaptive provers.

Athamnah, Ron-Zewi, Rothblum. Linear prover IOPs in log star rounds. IACR ePrint 2025/1269. [ePrint] - O(log* S) rounds; the GPU-phase relevance.

Setty, Thaler, Wahby. Customizable constraint systems for succinct arguments. IACR ePrint 2023/552. [ePrint] - CCS/SuperSpartan; the still-gated next-MLE item of STATUS section 8.

Justin Thaler. Sum-check is all you need: an opinionated survey on fast provers in SNARK design. IACR ePrint 2025/2041. [ePrint] - the checklist this report's Chapter 4 follows.

Wei et al. SNARKs over small prime fields without extension field multiplication. IACR ePrint 2025/719, ASIACRYPT 2026. [ePrint] - packed sumcheck via matrix-ring challenges (prime-field; noted for contrast).

Cluster C - commitments, accumulation, proof size

Gurkan, Novakovic, Rothblum. Bolt: faster SNARKs from sketched codes. IACR ePrint 2026/310, CRYPTO 2026. [ePrint] - (3+eps)N adds + (1+eps)N hashes; 5-15x smaller proofs than Brakedown.

Novakovic and Angeris. Ligerito: a small and concretely fast polynomial commitment scheme. IACR ePrint 2025/1187. [ePrint] - the MLPCS both Flock and BinarySpartan run on.

Peceny, Rachuri, Raghuraman, Rindal. SPIN: rate-1/2 codes with O(N) transposed encoding, Lean-verified. IACR ePrint 2026/2317. [ePrint].

Akhiani and Y. Zhang. RAA codes over large fields. IACR ePrint 2026/524. [ePrint] - relative distance 1/2 at n = 2^20 over large fields.

Arnon, Chiesa, Fenzi, Yogev. STIR: Reed-Solomon proximity testing with fewer queries. IACR ePrint 2024/390, CRYPTO 2024. [ePrint].

Arnon, Chiesa, Fenzi, Yogev. WHIR: a low-degree proof with tiny opening sizes. IACR ePrint 2024/1586, EUROCRYPT 2025. [ePrint] - 63 KiB proofs at degree 222, 360 microseconds verification.

Zeilberger, Chen, Fisch. Basefold: succinct arguments over random foldable codes. IACR ePrint 2023/1705, CRYPTO 2024. [ePrint].

Diamond and Posen. Succinct arguments over towers of binary fields (Binius). IACR ePrint 2023/1784, EUROCRYPT 2025. [ePrint].

Diamond and Posen. Polylogarithmic proofs for multilinears over binary towers (FRI-Binius). IACR ePrint 2024/504, EUROCRYPT 2026. [ePrint] - the ring-switching compiler plus char-2 Basefold.

B. E. Diamond. Zero-knowledge polynomial commitment in binary fields. IACR ePrint 2025/1015. [ePrint] - Aurora masking in char 2.

Zhang et al. LiftWHIR: DEEP sampling plus interleaving for the WHIR evaluation phase. IACR ePrint 2026/1561. [ePrint] - 4.4-6.7x eval-phase prover for 1.33-1.46x size.

Jia et al. ReedWeave: interleaved-and-folded RS commitments. IACR ePrint 2026/2147. [ePrint].

Bunz, Mishra, Nguyen, Wang. Accumulation without homomorphism. IACR ePrint 2024/474. [ePrint] - [BMNW24]: the bounded-depth spot-check accumulation our wrap.rs implements.

Bunz, Mishra, Nguyen, Wang. Arc: accumulation for Reed-Solomon codes. IACR ePrint 2024/1731. [ePrint] - unbounded RS accumulation via quotients and out-of-domain sampling.

Hadas Zeilberger. Khatam: proximity gaps for multilinear evaluation for all linear codes. IACR ePrint 2024/1843, CRYPTO 2026. [ePrint] - the theorem our fold chain needs.

Bunz, Chiesa, Fenzi, Wang. WARP: linear-time accumulation schemes. IACR ePrint 2025/753. [ePrint] - unbounded accumulation for any linear code over a sufficiently large field; the headline import.

Baweja, Mishra, Mopuri, Shtepel. FICS and FACS: linear-prover proximity IOPs and NP accumulation. IACR ePrint 2025/737. [ePrint].

Baweja, Mishra, Mopuri, Namboothiry. Vector commit-and-reduce schemes (Canopy). IACR ePrint 2026/2790; randomshuffle/canopy-system. [ePrint] - openings to proximity claims; -50% proofs on Bolt codes.

Guo, Liu, Huang, Qu, Tao, Zhang. DeepFold: efficient multilinear commitment from RS codes and list decoding. IACR ePrint 2024/1595, USENIX Security 2025. [ePrint] - batch evaluation for non-power-of-two sizes.

Wang and Zhang. UltraFold: packed Merkle trees for distributed Basefold. IACR ePrint 2026/266. [ePrint].

Arnon, Chiesa, Fenzi, Yogev. Interactive proofs for batch polynomial evaluation. IACR ePrint 2026/448. [ePrint] - m openings at constant overhead over one.

Bunz, Chen, Guan. It takes two: proofs of work for Fiat-Shamir. IACR ePrint 2026/2326. [ePrint] - grinding as soundness amplification.

Crites and Stewart. On Reed-Solomon proximity gaps conjectures. IACR ePrint 2025/2046. [ePrint] - the capacity-conjecture disproofs.

Chai and Fan. FRI soundness above the Johnson bound via threshold halving. IACR ePrint 2026/858. [ePrint] - the ~2x-query unconditional repair, Lean-verified, char-2 valid.

Dao, Kominers, Thaler, Zheng. Counterexamples to beyond-Johnson proximity gaps over binary fields. IACR ePrint 2026/2424. [ePrint] - the binary-domain constraint on our low-rate dials.

Fenzi and Sanso. Small-field hash-based SNARGs are less sound than conjectured. IACR ePrint 2025/2197. [ePrint].

Skatharoudis. SoK: hash-based polynomial commitments and low-degree tests: from FRI to Basefold, STIR, and WHIR. IACR ePrint 2026/1367. [ePrint] - the two-lineage systematization.

Nguyen and Setty. Neo and SuperNeo: lattice folding for CCS with pay-per-bit costs. IACR ePrint 2026/242. [ePrint] - the Horizon-3 fallback.

Osadnik. PikkuFold. IACR ePrint 2026/1809. [ePrint]; Osadnik and Seiler. LaBinius. IACR ePrint 2026/2103. [ePrint]; Bloemen, Garreta, Kostrzewa, Londhe, Soukhanov, Wu. BitZ. IACR ePrint 2026/2141. [ePrint].

Cluster D - hash circuits, memory, folding, stateless

Kostrzewa, Klein, Adkins, Swirski, Zmuda. Keccacheck: towards a SNARK friendly Keccak. IACR ePrint 2025/1764; reilabs.io blog; github.com/reilabs/keccacheck. [ePrint] - <4,000 constraints per permutation at batch >= 512.

Soukhanov. Hashcaster: GKR over binary fields with Frobenius unpacking. github.com/morgana-proofs/hashcaster. [web] - the Frobenius multiopen and boolcheck.

binius-zk. Binius64 and the binius.xyz benchmark tables; Irreducible's shutdown note (11/2025) and the MIT relicensing. [web] - word-level gates; ring-switching size data.

Grassi, Khovratovich, Koschatko, Rechberger, Schofnegger, Schroeppe, Wu. Poseidon(2)b: binary field versions of Poseidon/Poseidon2. IACR ePrint 2025/1893, IACR Commun. Cryptol. 2(4) 2026. [ePrint] - documented, not adopted.

Ashur, Mahzoun, Posen, Sijacic, Vision Mark-32: ZK-friendly hashing over binary tower fields. IACR ePrint 2024/633. [ePrint] - documented, not adopted.

Liu, Koschatko, Grassi, Yan, Chen, Banik, Meier. Opening the blackbox: collision attacks on round-reduced Tip5, Tip4, Tip4', and Monolith. IACR ePrint 2024/1900. [ePrint] - why the young designs stay on the shelf.

Drake, Khovratovich, Kudinov, Wagner. Hash-based multi-signatures for post-quantum Ethereum. IACR ePrint 2025/055. [ePrint] - the hash-based PQ context for our BLAKE3 posture.

Garreta, Hovhanissyan, Jivanyan, Manzur, Villalobos, Zajac. On amortization techniques for FRI-based SNARKs. IACR ePrint 2024/661. [ePrint] - packing and split-and-pack.

Ben-Sasson, Goldberg, Kopparty, Saraf. DEEP-FRI. arXiv 1903.12243, ITCS 2020. [web] - the out-of-domain sampling lineage.

Nair, Thaler, Zhu. Proving CPU executions in small space. IACR ePrint 2025/611. [ePrint] - the streaming shape our hierarchical root implements.

Hafez, Anthoine, Campanelli, Fiore. SoK: lookup table arguments. IACR ePrint 2025/1876. [ePrint] - the lookup map.

Soukhanov. Logup*: faster lookups for small-table indexed access. IACR ePrint 2025/946. [ePrint].

Bunz, J. Chen, DeStefano, B. Chen. Almost linear-time permutation check. IACR ePrint 2025/1850. [ePrint] - MulPerm/BiPerm; the small-field soundness fix.

Beal and Fisch. Mira: folding for pairing-based arguments. IACR ePrint 2024/2025. [ePrint] - pairing-bound; the negative result for hash-based folding.

Kothapalli and Setty. CycleFold. IACR ePrint 2023/1192. [ePrint]; Bunz and Chen. ProtoStar. IACR ePrint 2023/620. [ePrint]; Kothapalli, Setty, Tzialla. SuperNova. IACR ePrint 2022/1758. [ePrint] - the EC folding lineage, structurally inapplicable here.

Proof-carrying data via holography accumulation. IACR ePrint 2026/538. [ePrint]; Zero-knowledge PCD from accumulation schemes. IACR ePrint 2026/289. [ePrint]; On the simulation-extractability of PCD. IACR ePrint 2025/2037. [ePrint].

Tas and Boneh. Vector commitments with efficient updates. IACR ePrint 2023/1830. [ePrint]; Bender, Kuszmaul, Kuszmaul. Faster incremental authenticated dictionaries. IACR ePrint 2026/2098. [ePrint]; Pathak and Ruj. Post-quantum vector commitments with efficient insertions and deletions. IACR ePrint 2026/2592. [ePrint]; BORG: extendable distributed VCs. IACR ePrint 2026/1552. [ePrint].

ethereum.org. The statelessness roadmap page (updated 10/2026). [web] - weak statelessness requires Verkle + PBS; zkEVM-L1 verification complementary.

Irreducible. Ethereum state proving service benchmarks (01/2026 blog). [web] - 1,419 account reads: Binius 11-28 s CPU vs SP1 ~100M cycles; the keccak-MPT cost reference.

Srinath Setty. Magnetar: high-speed post-quantum RV64 zkVM from standard hashes. IACR ePrint 2026/2423. [ePrint] - the paper this repository implements; Table 6 is our target.

Mingshu Cong. Mikros: bitwidth-aware SNARKs over binary fields. IACR ePrint 2026/2646. [ePrint] - implemented in magnetar-mikros; the local PDF is upload/2026-2646.pdf.

Srinath Setty. BinarySpartan: Spartan over binary fields. IACR ePrint 2026/1656. [ePrint] - implemented in magnetar-spartan and magnetar-mikros::spartan.

Goldwasser and Kalai et al. Incrementally verifiable oracle-aided computation. IACR ePrint 2026/2824. [ePrint] - the updatable-forest/signed-oracle plan (repo W1-W12); not re-researched here.

This repository. docs/BENCHMARKS.md, docs/ROADMAP.md, docs/RECURSION.md, docs/STATUS.md, docs/PERFORMANCE.md, docs/ORACLE_AIDED_IVC.md. [repo] - the measured baselines and the standing plans this report extends.